Software Engineering Department
Braude College of Engineering
Final Project – Phase B (61999)

FlowGrid: Adaptive Traffic Signal Control Using Reinforcement Learning

Team Code: 26-1-D-30

Students: Avishag Levi, Einav Momi Ben Shushan

Advisor: Dr. Cohen Reuven

Git Repository: https://github.com/einavbs1/TrafficProject

Demo Video: https://drive.google.com/file/d/1BE3oeGWWbVrQEC_ZL0rVh5AdPs9kDNTY/view

Table of Contents

1. Introduction

1.1 The Problem

Most traffic signals in use today run a fixed cycle of green, yellow, and red, timed once by an engineer and rarely revised. A fixed schedule works while demand stays near the historical averages from which it was derived. Real demand, however, is highly dynamic. It rises during peak hours, falls overnight, and responds to accidents, weather, and events that a fixed timer cannot detect. The result is a controller that allocates green time to an empty approach while a congested one waits, simply because it cannot differentiate between them.

Adaptive controllers address part of this gap. Vehicle-actuated control improved upon fixed timing via inductive loop detectors, extending or terminating a phase according to vehicle presence. These systems rely on binary presence detection rather than volumetric measurement, and their sensing range extends only a short distance past the stop line, rendering them unable to determine queue lengths. More importantly, none of them learn. Their underlying logic is fixed in advance by an engineer, and only the inputs change.

1.2 Research Questions

This project asks three questions:

- Can a controller given no explicit switching rules, learning only from repeated simulated experience, outperform the fixed-time signals already in widespread use?
- Under which traffic conditions does it succeed, where does it fall short, and what explains the difference?
- How much confidence can we place in that answer, given that a reinforcement learning agent can exploit its own reward function without providing tangible benefits to road users?

1.3 Our Solution

This project aims to build a reinforcement learning agent that controls a single traffic signal intersection more effectively than standard deployed alternatives, and to evaluate it rigorously. In this context, enhanced effectiveness is defined as a measurable reduction in total vehicle waiting time across light, moderate, and heavy traffic conditions.

The final system, named FlowGrid, consists of three subsystems.

The first is a SUMO-based simulation environment. It models a four-way intersection with randomly generated vehicle demand, a camera with a limited sensing range (the agent observes only 150 meters back from the stop line, representing plausible roadside sensor capabilities), and a set of hard safety constraints, including minimum and maximum green durations and mandatory yellow transitions.

The second is the trained agent: a Proximal Policy Optimization (PPO) agent, trained from scratch with no predefined switching rules, that decides every five simulated seconds whether to hold the current phase or advance to the next.

The third is an evaluation and demonstration layer. This includes two applications, a suite of statistical comparison tools, and a Watch Live mode that opens a native SUMO visualization window, allowing the agent's performance to be interactively evaluated and verified.

1.4 How We Tested It, and Against What

Every method in this project is compared against fixed-time control, the dominant approach currently deployed at most intersections, using cycle lengths of 30, 45, and 60 seconds as three separate baselines. We also considered including Max Pressure, a principled adaptive controller from the traffic engineering literature; however, its integration was ultimately excluded from the current scope to maintain a strict focus on evaluating the agent against the most widely deployed fixed-time baselines.

Every comparison is executed under identical traffic conditions. SUMO's vehicle generation is seeded, meaning a given seed produces an identical stream of vehicles regardless of the active controller. Consequently, any divergence in outcomes is strictly attributable to the controller's policy rather than variations in traffic difficulty. We tested all three demand levels, and the final methodology evaluates every saved checkpoint against its own independent seed and scenario, reporting every outcome rather than a selected best run.

1.5 Who This Book Is For

The intended stakeholders fall into two categories. For traffic engineers and municipal decision-makers, the primary focus is the evaluation results: does the method outperform what is already in use, under which conditions, and with how much confidence? The second group comprises researchers aiming to build upon this work; for them, the comprehensive version history, transparent documentation of failed iterations, and detailed architectural rationales hold primary value.

2. Background, Tools and Methods

2.1 Related Work

Traffic signal control has evolved through several generations. Fixed-time control remains the most prevalent approach, operating on a rigid schedule derived from historical traffic counts (Koonce et al., 2008) and lacking the capacity to adapt to real-time conditions. Vehicle-actuated control improved upon this framework by utilizing inductive loop detectors to extend or terminate a phase based on vehicle presence. However, a primary limitation of these systems is their reliance on binary presence detection rather than volumetric measurement; consequently, a single vehicle on a minor approach can disrupt a heavy flow on a main arterial (Akçelik, 1994).

A subsequent generation moved beyond a single intersection's local logic. SCOOT models platoons using upstream detectors and continuously adjusts split, cycle, and offset to maintain a coordinated green wave along a corridor (UK Department of Transport, 1995). SCATS, conversely, pursues equisaturation by measuring the Degree of Saturation at the stop line and reallocating green time to balance competing approaches (Akçelik, 2010). While both methods are genuinely adaptive and widely deployed, their underlying algorithmic logic remains pre-defined, rendering them strictly reactive to live inputs rather than capable of learning.

A more modern line of research focuses on signal-free infrastructure. Dresner and Stone (2008) proposed a reservation-based Autonomous Intersection Management protocol, in which

vehicles negotiate directly with an intersection manager to pass through without stopping. Liang et al. (2018) demonstrated a Vehicle-to-Infrastructure architecture, where vehicles broadcast their position and intent to roadside equipment, feeding a high-resolution grid into a Double Dueling Deep Q-Network. However, both methods depend on a level of connected vehicle network penetration that has not yet been realized in real-world deployment.

The original design was built on three methods:

- **YOLO (You Only Look Once)** (Redmon et al., 2016) is a highly efficient object detection algorithm. Rather than scanning multiple candidate regions sequentially, it processes the entire image in a single pass to draw bounding boxes and classify objects, making it ideal for live video feeds. Its intended role in our design was to identify vehicles on each approach from a camera feed, producing a lane-specific volume count as a primary input for the controller.
- **DeepSORT** (Wojke et al., 2017) is a tracking algorithm that operates in conjunction with a frame detector like YOLO. While frame detectors evaluate images independently, DeepSORT links detections across frames into continuous tracks by predicting future positions and matching visual appearances, ensuring identity persistence even during temporary occlusions. Its purpose was to assign persistent identities to vehicles, enabling the calculation of individual waiting times. This per-vehicle waiting time forms the basis of the starvation signal in our observation and reward mechanisms (Section 3.3), a metric that static detection alone cannot provide.
- **Deep Q-Network (DQN)** (Mnih et al., 2015) is a reinforcement learning algorithm that utilizes a neural network to estimate the future value of available actions in a given state, selecting the optimal action and refining its estimates through experience replay. The Double Dueling variant, utilized by Liang et al., incorporates refinements that mitigate the original algorithm's tendency to overestimate action values. Its role was to execute the core decision-making loop, determining every few seconds whether to hold or advance the phase. While we initially implemented this component of the pipeline, Section 4.3 details our subsequent transition to a different algorithmic approach.

Our original proposal aimed to utilize a roadside camera as the primary sensor, combining these three methods into a comprehensive pipeline designed to be retrofitted onto existing intersections. Wang et al. (2024) recently validated a closely related framework, confirming this remains a highly active research direction. This intended architecture also explains why our agent's observation range is constrained to 150 meters back from the stop line. The observation space was explicitly shaped to mimic plausible roadside camera capabilities, ensuring the trained policy could theoretically integrate with a real-world perception pipeline. Although the vision layer was ultimately excluded from the delivered scope (as explained in Chapter 4), it remains the natural next step for future work.

2.2 Why Reinforcement Learning Fits This Problem

The traffic control problem maps directly onto the reinforcement learning formalism. An agent (the signal controller) observes a state describing current demand at each approach, executes an action, and receives a reward indicating whether the decision improved traffic conditions. This cycle repeats across thousands of simulated episodes. However, three primary challenges complicate this application. First, optimal reward formulation is non-trivial, as minimizing waiting time, maximizing throughput, and maintaining fairness across approaches are distinct

and sometimes conflicting objectives. Second, the environment is only partially observable, as realistic roadside sensors have limited upstream visibility. Third, an agent can artificially inflate its reward score by exploiting the reward definition without generating tangible benefits for road users, a phenomenon we encountered repeatedly during development.

To isolate and address these challenges systematically, we developed a dedicated single-intersection environment in SUMO (Simulation of Urban Mobility) before attempting a larger network. Reward design, partial observability, the exploration-exploitation trade-off, and the risk of reward exploitation are all present at a single four-phase intersection and are significantly easier to diagnose in this controlled setting.

2.3 Tools and Technologies Used

The simulation is powered by SUMO, accessed via Python through the sumo-rl and TraCI/libsumo interfaces. Training was conducted using Stable-Baselines3, specifically leveraging its sb3-contrib extension to enable the maskable variant of PPO, deployed over vectorized parallel environments running ten independent SUMO instances simultaneously. Evaluation and statistical analysis were performed utilizing pandas, matplotlib, and seaborn.

The interactive evaluation tooling was tailored to the specific agent. The PPO agent is supported by a FastAPI application featuring a custom-built HTML, CSS, and JavaScript front end that includes a guided user tour. The DQN agent utilizes a Tkinter-based desktop application. Additionally, FlowGrid_Web, detailed in Section 3.7, provides a React and Vite front end over an independent FastAPI backend. Throughout the project, all software components were intentionally designed to maintain minimal external dependencies rather than relying on monolithic frameworks.

3. The Delivered System

This chapter presents the algorithms and methodology behind the agent. The software that implements them is documented in the Developer and Maintenance Guide (Appendix B).

3.1 System Architecture

FlowGrid is a closed loop training system. A simulated intersection advances in fixed time steps, the agent chooses a control action at each step, and the resulting change in traffic state becomes a reward that updates the policy. Training repeats this loop for several million steps. The components below exist to run that loop correctly and fast enough to be practical.

The architecture has three layers: the simulator that models traffic, a translation layer that converts raw simulator state into a learnable representation, and the learning algorithm. The agent never talks to the simulator directly. Every observation it receives and every action it issues passes through the middle layer. That separation is what made the sequence of reward and observation experiments in Section 4.4 practical, since each one changed a single layer without disturbing the others.

Figure 3.1 shows these layers as a UML component diagram. Every component named in it is a real class or module in the delivered code.

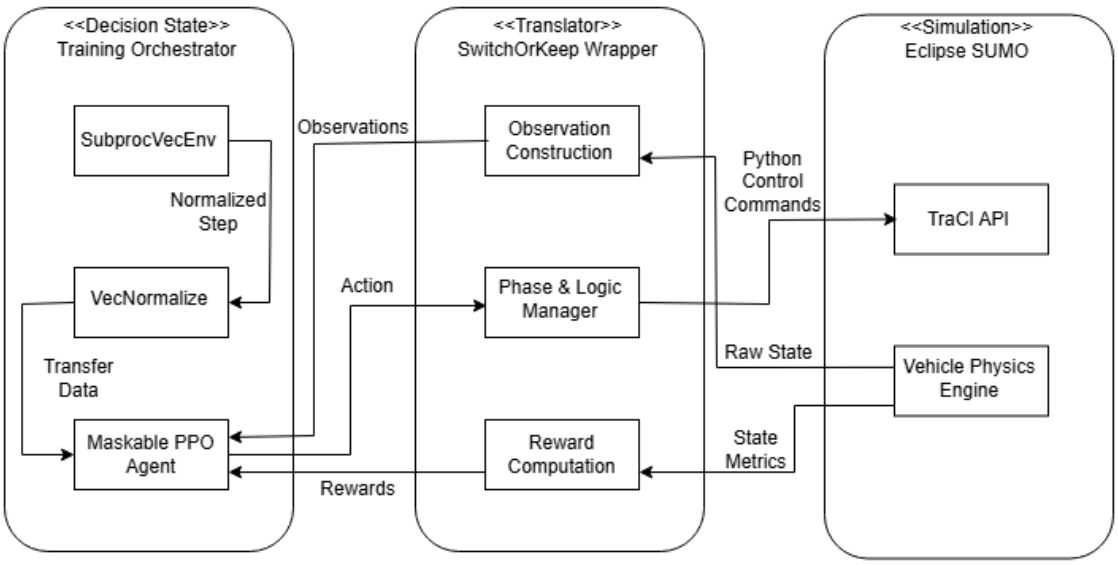


Figure 3.1: The training and simulation architecture.

Working from the bottom of the diagram upward:

- **Simulation layer:** Eclipse SUMO models the road network, vehicle dynamics, car following, and queue formation, and holds the ground truth state: where every vehicle is and how long it has waited. It is operated via TraCI, SUMO's remote control interface, which allows an external process to read state metrics and set signal phases during runtime. One critical configuration choice is noteworthy: SUMO by default removes vehicles that have been stationary for an implausibly long time. We disabled this feature, as it allowed an early version of the agent to exploit the reward function (Section 7.1).
- **Translation layer:** The SwitchOrKeepWrapper sits between the simulator, which deals in individual vehicles on individual lanes, and the learning algorithm, which needs a fixed-length observation vector and a discrete action. This layer serves three primary functions. It builds the 21-element observation described in Section 3.3, subject to the 150-meter sensing limit. It enforces the constraints the policy may not override: a minimum green of 10 seconds, a maximum of 60, a mandatory 3-second yellow on every phase change, and an action mask that removes Switch when no vehicle is present. Additionally, it computes the reward. The majority of architectural iterations detailed in Section 4.4 were confined strictly to this layer.
- **Parallel environment execution:** Training duration is strictly bound by the rate of experience generation, and a single simulation instance generates data too slowly. SubprocVecEnv, a standard Stable-Baselines3 component, runs ten SUMO instances in separate processes, each with independently generated demand, and merges their experience into one training stream. This parallelization reduces the duration of a six-million-step training run from days to hours.
- **Observation normalization:** The 21 observation elements span significantly different numeric scales, and gradient-based training degrades when one input is far larger than the rest. VecNormalize tracks a running mean and variance for each element and rescales incoming observations against them, clipping to bound outliers. We normalize observations but not rewards, since the reward is already a bounded physical quantity. These statistics are part of the trained model, not a side file: a checkpoint restored

without them receives inputs on the wrong scale and behaves incorrectly, necessitating the serialization of these normalization statistics alongside each saved model checkpoint.

- **Policy optimization:** The MaskablePPO agent encapsulates the policy, utilizing a multilayer perceptron with two hidden layers of 128 units for the policy head and two for the value head. Action masking is a critical architectural feature of this policy. At each decision, the translation layer supplies the set of legal actions, and illegal ones are removed from the policy distribution before sampling rather than rejected afterwards. Section 3.4 details the significance of this mechanism.

A single decision step operates as follows. The simulation advances five simulated seconds. The translation layer reads the resulting state through TraCI, builds the 21 observation elements, computes the reward earned by the previous action, and determines the current action mask. The observation is normalized. The policy selects either Keep or Switch. The translation layer converts this selection into signal commands, inserting the yellow interval on a phase change. The cycle repeats. Training executes this loop across ten parallel environments until the step budget is exhausted. Evaluation follows the identical loop with policy updates disabled, driven by the tooling described in Section 3.6.

3.2 The Decision Problem, Formally

Every reinforcement learning problem is fundamentally formalized as a Markov Decision Process (MDP), comprising a state space, an action space, a reward function, and transition dynamics governing state evolution. Structuring the traffic control problem within this formal framework ensures rigorous definitions for parameters that might otherwise remain ambiguous. The specific implementations of these core MDP components within our architecture are detailed directly in Section 3.3.

3.3 State, Action, and Reward

The state is a 21-element vector, normalized to the range of 0 to 1: a one-hot encoding of the currently active green phase, the elapsed green time as a fraction of its maximum allowance, an occupancy density for each of the eight lane groups, and a starvation score per lane group indicating the waiting time of the longest-delayed vehicle. The starvation metric is critical because an intersection might exhibit a reasonable average wait time while individual vehicles in rarely served lanes experience unacceptable delays. This term ensures the agent addresses severe individual delays even when aggregate metrics appear optimal.

The action space is intentionally constrained: the agent may only hold the current phase or switch to the next in a fixed cyclic order. Transitions to arbitrary phases are prohibited. Early experiments (Chapter 7) demonstrated that an unrestricted action space encouraged rapid cycling through yellow transitions; imposing a cyclic structure eliminates this failure mode by design. The hard constraints listed in Section 3.1 operate on top of this action space and are strictly enforced outside the policy network.

The reward function is defined as the reduction in total system-wide waiting time between decision steps, measured directly from the simulator, minus a minor penalty tied to the highest starvation score. This streamlined reward structure was adopted following extensive iterations. An earlier reward formulation (Chapter 7) utilized numerous hand-tuned terms, making it exceedingly difficult to isolate the specific incentives driving observed behaviors. Aligning the

reward directly with the primary evaluation metric significantly improved training stability and simplified debugging.

3.4 Why MaskablePPO

The final agent was trained using MaskablePPO, an extension of Proximal Policy Optimization that incorporates native support for action masking. When an action is structurally disallowed, such as switching phases at an empty intersection, its probability is eliminated from the policy distribution prior to sampling, rather than being rejected post-hoc. While Deep Q-Networks (DQN) were also considered and an earlier implementation exists in the codebase (Section 4.3), PPO was ultimately selected for its stability. Specifically, PPO's clipped objective function and early stopping mechanism based on excessive KL divergence provided two independent safeguards against the abrupt policy collapses encountered in earlier development phases (Section 7.2). In practice, these stabilization mechanisms proved more valuable for our architecture than the theoretical sample efficiency of off-policy methods.

3.5 The Decision Loop in Operation

Figure 3.2 traces the decision loop as a UML activity diagram, representing the current operational architecture. Each node corresponds to a specific programmatic step within sumo_rl_env_V8.py. Two key aspects of this flow should be highlighted. First, the hard action mask, the logical step that removes the "Switch" action, functions as the structural constraint introduced in Section 3.3, applied preemptively rather than as a posterior penalty. Second, to clarify the system's operational boundaries, this loop relies exclusively on the simulated state metrics. Advanced conceptual components such as YOLO detection, DeepSORT tracking, Priority Protocol, and Fallback Mode are deliberately absent from this diagram, as their implementation was excluded from the final delivered scope (as detailed in Chapter 4).

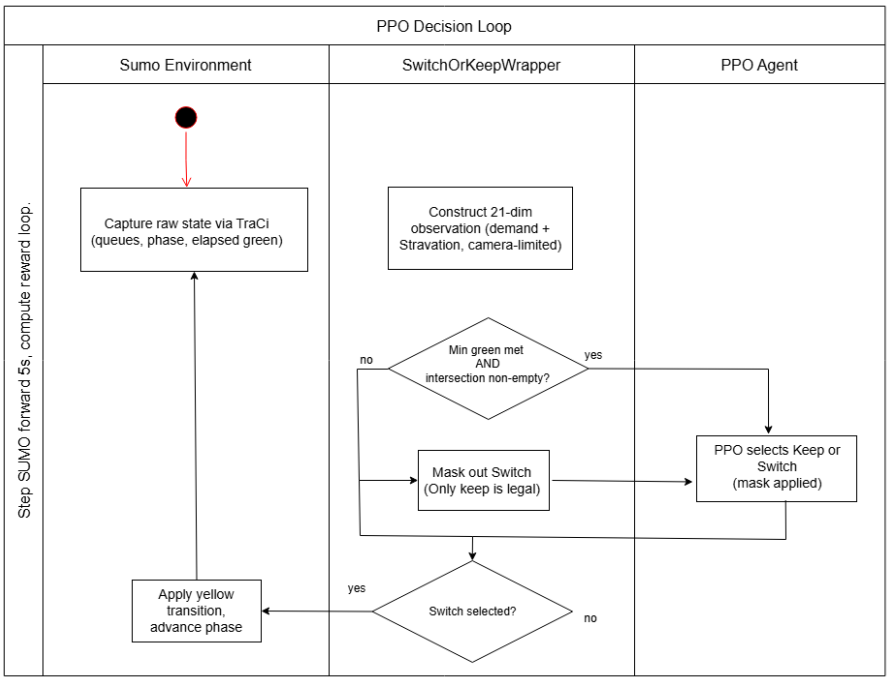


Figure 3.2: Activity diagram of the decision loop.

3.6 The Comparison Application

The comparison application facilitates independent verification of the agent's performance and safety metrics; a detailed evaluation of these claims is presented in Chapter 5. The interface allows users to select a trained model, baseline comparators, a specific traffic scenario, and random seeds to execute the comparison directly within a browser. Furthermore, its Watch Live mode launches a native SUMO graphical window, enabling qualitative observation of the agent's behavior rather than relying solely on numerical inference. Designed for accessibility, the application features an interactive, step-by-step guided tour that sequentially highlights each input field. This guide explains key functionalities, such as model selection, baseline configuration, seed and scenario generation, and the Watch Live feature. The tour initializes automatically upon the first visit and can be relaunched on demand via a dedicated button in the header.

Figure 3.3 illustrates the output of the Watch Live mode, displaying SUMO's native graphical interface as seen during live demonstrations. The visualization accurately represents the intersection geometry and dynamically generated vehicles (shown in yellow), while the red and green bars at each approach indicate the active signal phases.

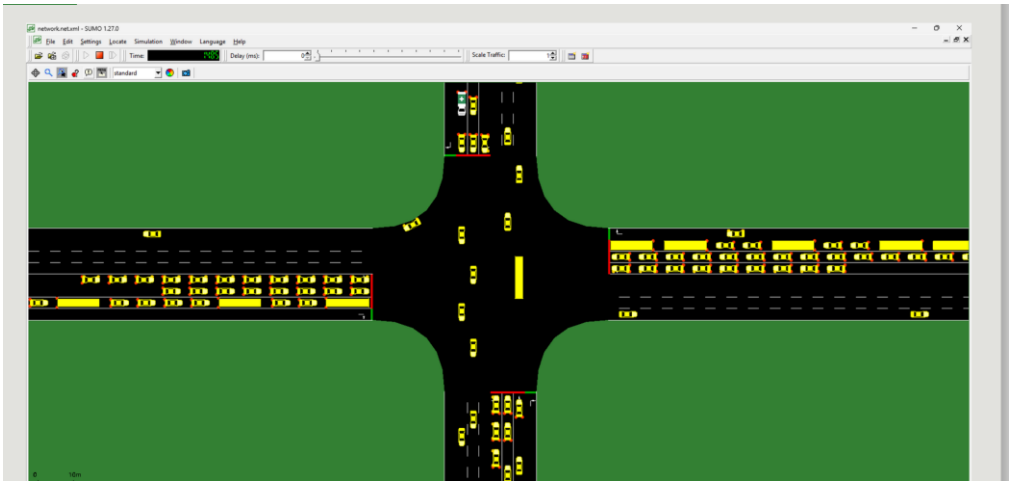


Figure 3.3: The SUMO window during a Watch Live session.

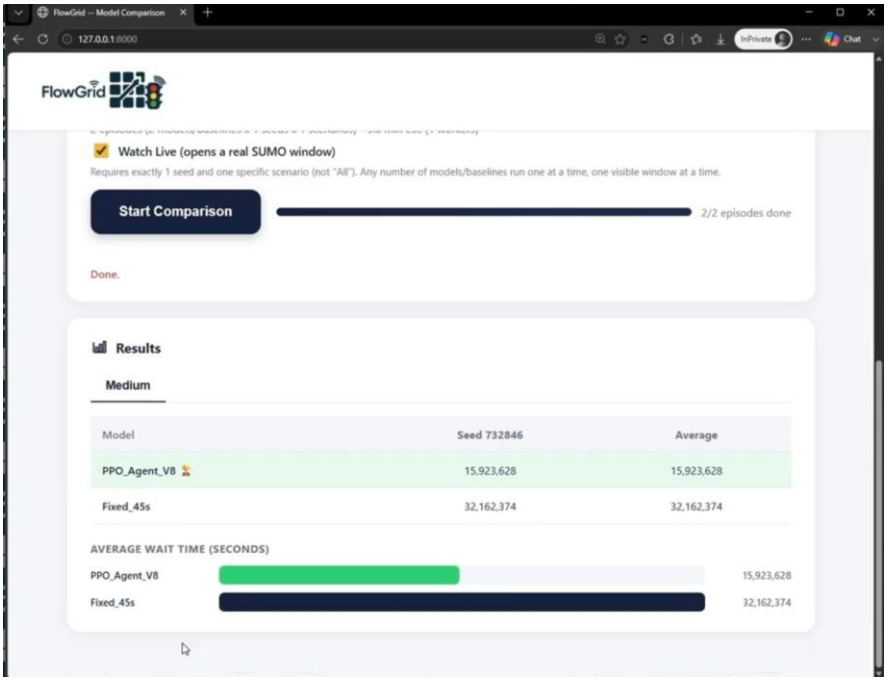


Figure 3.4: The comparison web application, mid comparison.

Figure 3.4 displays the application interface upon the completion of an evaluation episode, comparing the final PPO_Agent_V8 model against the Fixed_45s baseline under moderate traffic conditions using seed 732846. The interface presents the aggregated performance metrics through a generated data table and a corresponding graphical bar chart.

3.7 FlowGrid_Web, the Operations Dashboard

This phase also delivered FlowGrid_Web, a cloud-based traffic operations dashboard featuring an independent backend, multi-junction navigation, user authentication, device configuration, and reporting capabilities. This application demonstrates how a fully deployed system would be presented to a municipal traffic authority. While the user interface is fully functional, the underlying data is simulated for the majority of the junctions to serve as a structural template. The deliberate exception is the "Live Junction" module, which is directly integrated with the trained PPO agent and the SUMO simulator utilized throughout this study. Triggering the "Run Agent" control launches a live SUMO episode, streaming real-time queue counts per direction, active signal phases, and a live snapshot from the simulation window (as previously shown in Figure 3.3).

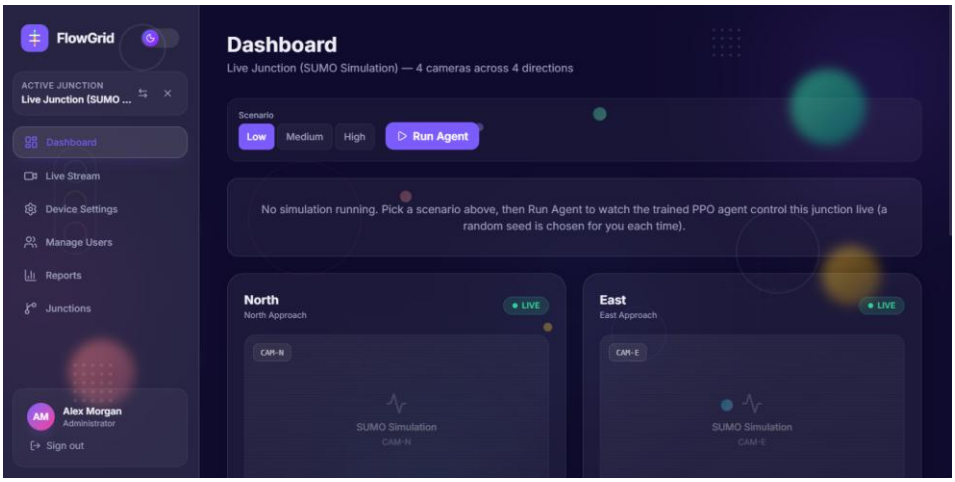


Figure 3.5: The FlowGrid_Web operations dashboard, displaying the Live Junction interface, scenario selection, and simulation controls.

While the formal Use Case Diagram is documented in an earlier architectural section, FlowGrid_Web successfully implements all of its defined use cases as interactive application views. The system distinguishes between standard User and Administrator roles, facilitating the following core operational processes:

- **Log In:** Provides role-based access control. In this demonstration environment, it accepts predefined credentials rather than querying a live authentication server.
- **View Junction Data:** Displays operational metrics, video snapshots, and current traffic light states. As noted, this data stream is live and driven by the RL agent for the designated SUMO junction, while mocked elsewhere.
- **Manage Devices Settings & Switch Between Modes:** Allows operators to toggle intersection operational modes and adjust simulated hardware configurations.

- **Configure Direction & Traffic Signals (Admin):** Enables administrators to define intersection geometries and signal logic, a process which inherently incorporates the workflow to Add a Junction.
- **View Reports & Manage Users (Admin):** Provides administrative oversight through system-wide performance reporting and user access management.

By translating these theoretical use cases into a functional interface, FlowGrid_Web bridges the gap between the reinforcement learning backend and a practical, user-facing traffic management product.

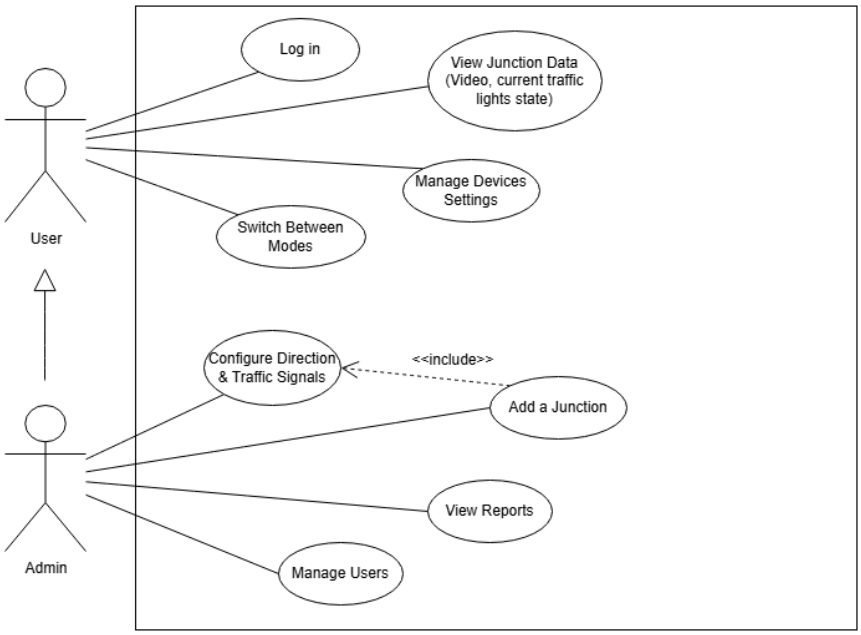


Figure 3.6: Use Case Diagram from the Phase A proposal.

A recorded video walkthrough of the system, including a live Watch Live session and the comparison app end-to-end, is provided alongside this documentation.

4. Development Process and Decision Making

4.1 From the Original Plan to What We Actually Built

The delivered system represents a refined and more rigorously evaluated iteration of the Phase A proposal. The initial proposal outlined a highly expansive scope, encompassing a full YOLO and DeepSORT computer vision pipeline, a rule-based emergency Priority Protocol, and a Deep Q-Network (DQN) agent for phase selection. However, to maximize algorithmic reliability and evaluation rigor, the project deliberately narrowed its focus. The computer vision and emergency priority layers were excluded from the final scope, allowing us to concentrate entirely on training, stabilizing, and rigorously verifying the core control policy using the simulator's ground-truth vehicle state.

Figure 4.1 illustrates the updated deployment architecture of the delivered system. The overall structural design, including the cloud-hosted infrastructure (FastAPI backend and PostgreSQL database), the edge computing framework, and the communication protocols, remains functionally consistent with our original plan and is actively demonstrated in the FlowGrid_Web dashboard. The primary architectural revision reflected in this diagram is the transition from the

initially proposed DQN agent to a Maskable Proximal Policy Optimization (PPO) agent. This pivotal shift was executed to leverage PPO's superior training stability and its native capability for structural action masking, resolving critical performance bottlenecks encountered during early development.

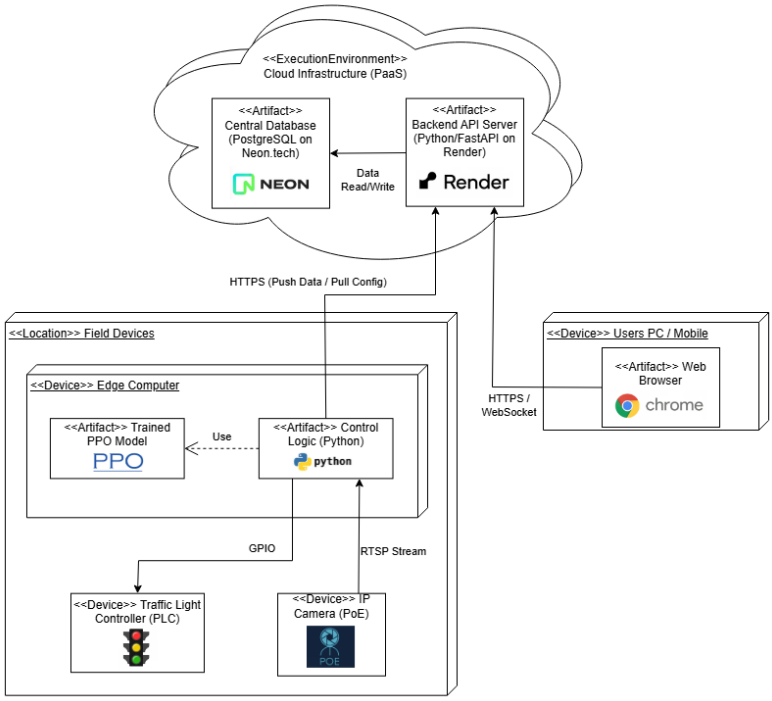


Figure 4.1: The updated deployment architecture, reflecting the transition to a trained PPO agent within the planned infrastructure.

4.2 Why the Scope Narrowed

Two factors explain this. The first was a deliberate research decision. Deciding which phase should be green and for how long is already a substantial problem on its own, as the version history in Section 4.4 shows. Combining it with an unsolved detection and tracking problem would have made it much harder to tell whether a poor result came from perception or from decision making. Isolating the control problem, with ground truth state standing in for perfect perception, let us study it on its own.

The second factor was circumstantial. During the project both team members were called up for military reserves, which is compulsory once summoned and cannot be deferred, removing a significant and unpredictable block of development time without notice. Faced with a real reduction in available time, we chose to protect the depth of the reinforcement learning work already underway rather than spread the remaining time across perception, tracking, priority logic, and physical hardware deployment, and finish none of them to a demonstrable standard. We see the work as two phases of the same project: the decision-making component, which we completed and evaluated rigorously, and the simulation-to-reality deployment layer, which remains the next phase rather than an abandoned attempt.

4.3 Changing the Algorithm, DQN to PPO

We also changed which reinforcement learning algorithm served as the primary approach. An earlier DQN implementation exists in the codebase, working on the same ground truth state

rather than vision input, with its own environment, a much more complex multi-term reward, and its own training pipeline.

The log demonstrates not simply that DQN was inferior to PPO, but that its performance varied too drastically between runs to be dependable. Thirty-four logged runs exist, excluding a thirty-fifth synthetic sanity check, all on the same fixed seed and spanning three intersection configurations over roughly three weeks. Improvement over the baseline ranged from effectively eliminating waiting time (99.6% better) to catastrophic failures where the agent caused severe gridlock and unacceptably long waiting times. While the median run showed a respectable 34.8% improvement, the mean was severely skewed by these extreme outliers. A metric where the mean and median diverge so drastically indicates a highly unstable controller.

Figure 4.2 plots every logged evaluation of the DQN implementation in chronological order. Each dot represents a real test at a different point in development, with the vertical axis showing the percentage improvement over the fixed-time baseline. To accommodate severe outliers without distorting the scale, the graph utilizes a symmetric logarithmic scale (symlog), placing catastrophic failures at the lower boundary. Blue points indicate performance superior to the baseline, red points indicate inferior performance, and gray points represent runs logged with zero waiting time, which are treated as failed evaluations rather than optimal results. Good outcomes and severe failures sit side by side throughout. Due to this extreme instability, we abandoned DQN and selected PPO as our final, stable version, which demonstrates the consistent pattern detailed in Chapter 5.

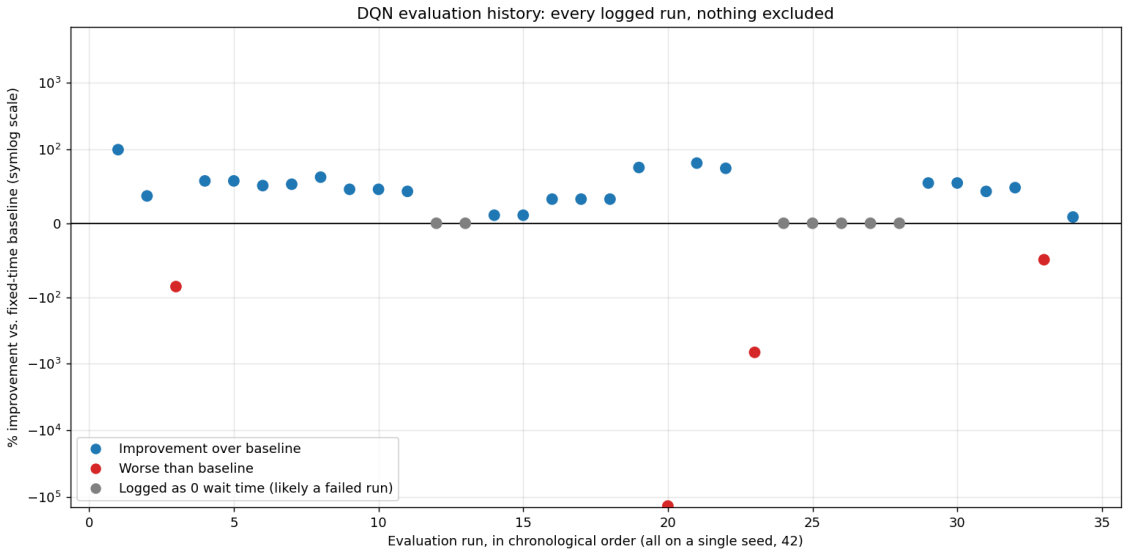


Figure 4.2: DQN evaluations against the fixed-time baseline, in chronological order.

By count, 23 of the 34 runs beat the baseline, 4 performed worse, and 7 were logged with zero waiting time. The worst cases represent genuine gridlock, such as one run where 411 vehicles were stuck on the map at the time limit, rather than a numerical artifact. We treat the seven zero-waiting-time entries as failed evaluations, not successes. No run was excluded from the chart or from these totals. This failure mode recurred frequently: individual approaches were starved almost entirely in favor of others, with nothing in the agent's internal logic to recover, despite more than a dozen hand-tuned reward terms designed to prevent precisely this behavior. PPO's clipped updates proved far more resistant to the abrupt policy collapse detailed further in Chapter 7, and its action masking allowed us to rule out unsafe behavior structurally

rather than through penalization. The DQN implementation remains fully accessible within the repository.

4.4 The Version History

Development did not proceed in a straight line, and the actual chronological sequence of iterations is reported here without simplification. Consequently, the version numbers in the table appear non-sequential, reflecting our true experimental path where we frequently reverted to earlier architectural branches (such as rolling back from V6 to V3) after encountering dead ends. Each listed version represents a training run initialized from scratch rather than a continuation from a previous checkpoint, as altering reward or observation definitions mid-training and resuming from an existing checkpoint corrupted the policy instead of improving it.

Version	Core idea tried	Outcome
V1 – V3	Cyclic phases, reward tied to raw waiting time	Agent exploited SUMO's vehicle teleportation feature to make waiting time vanish without serving anyone
V4 (Acyclic)	Let the agent jump to any phase directly	Excessive switching wasted enormous time in mandatory yellow transitions
V5	Pressure style reward with an uncapped exponential starvation penalty	Extreme penalty values exploded training gradients
V6 (capped)	Same idea, penalty capped	More stable, but still worse than a fixed time controller overall
V3.1 – V3.3	Reverted to cyclic phases. Dynamic minimum green rule added, then extended	V3.3 fully collapsed: a broken dynamic green guard was reinforced over two million additional steps until it became the agent's entire strategy
V4 (fresh)	Reward tied directly to the change in total waiting time	First version to beat any baseline at all, but still lost badly in light traffic
V6 (camera) / V7	Added a starvation penalty and a distance limited "camera" observation	Fixed the light traffic case. Introduced and then fixed a heavy traffic regression as camera range was tuned
V8, champion	Replaced a soft idle switching penalty with a hard rule: switching is structurally blocked at an empty intersection	First agent to beat all three fixed time baselines across all three traffic conditions
V9	Tested whether a richer, less saturating observation would fix the remaining heavy traffic gap	It did not, the gap turned out to be a demand ceiling, not a perception problem (Section 5.3)

The project version history documentation records every incremental change and its underlying rationale, while the table above provides the condensed summary.

4.5 How We Worked

The project was steered through frequent review sessions in which we demonstrated the agent live. This is the primary reason the graphical comparison tools exist: selecting a model, a scenario, and a seed, including one supplied on the spot, and observing the outcome immediately provided intuitive insights into the agent's actual behavior. However, because visual observation alone is insufficient for formal documentation, these live demonstrations were systematically backed by comprehensive numerical results and data tables to ensure rigorous, verifiable evaluation. Once V8 outperformed every baseline, the emphasis shifted from

improving performance to establishing how confidently that performance could be claimed. Chapter 6 documents the evaluation process that followed.

5. Results

5.1 The Headline Result

The project's central goal, developing an agent that outperforms fixed-time control across light, moderate, and heavy traffic, was achieved and independently verified. Our champion agent reduces total waiting time by roughly 50% in light traffic, 36% in moderate traffic, and 8% in heavy traffic, each measured against the best fixed-time baseline for that condition. Under the most rigorous test we conducted, described in Chapter 6, it outperformed all three baselines in 53 of 60 evaluation cases (88.3%). Its remaining losses were narrow, single-digit margins against the toughest baseline in light traffic, rather than systemic failures.

5.2 The Evidence

The following three figures illustrate this evidence directly. In each chart, our agent is evaluated against ordinary fixed-time signals, where lower values indicate less waiting time. Consequently, a curve positioned below the fixed-timer baselines signifies superior performance.

Figure 5.1 plots total waiting time across every checkpoint of our champion agent against the three baselines, utilizing one independently drawn seed and scenario per checkpoint. The logarithmic y-axis allows light and heavy traffic conditions to be visualized on a single chart. The agent begins with wait times comparable to or higher than the fixed timers, drops sharply within the first few hundred thousand steps, and remains below all three baselines for the remainder of training across light, moderate, and heavy traffic (represented by solid, dashed, and dotted lines respectively).

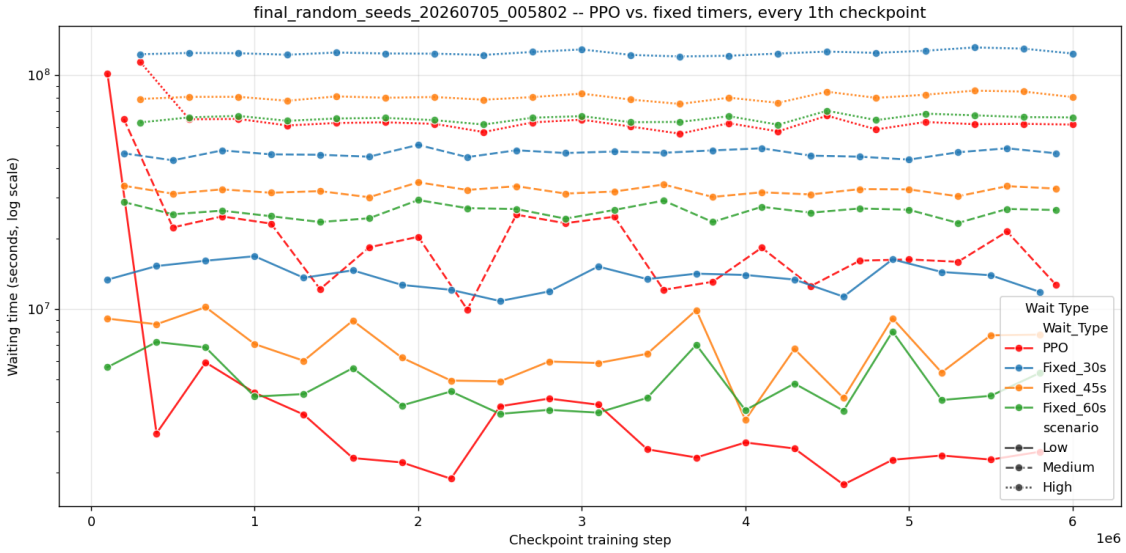


Figure 5.1: Total waiting time per checkpoint, against three fixed time baselines.

Figure 5.2 condenses these 60 checkpoints into a single metric per checkpoint: the percentage improvement over the Fixed_60s baseline, segmented by traffic condition. A data point positioned above the zero line indicates that the agent outperformed the baseline at that training stage, while a point below indicates underperformance. Aside from extreme negative outliers in the initial stages of training, nearly every point resides above the zero line. This

provides direct empirical evidence for the 88.3% win rate. The sharp contrast with Figure 4.2, where underperforming runs recurred frequently, highlights the stability of the final approach.

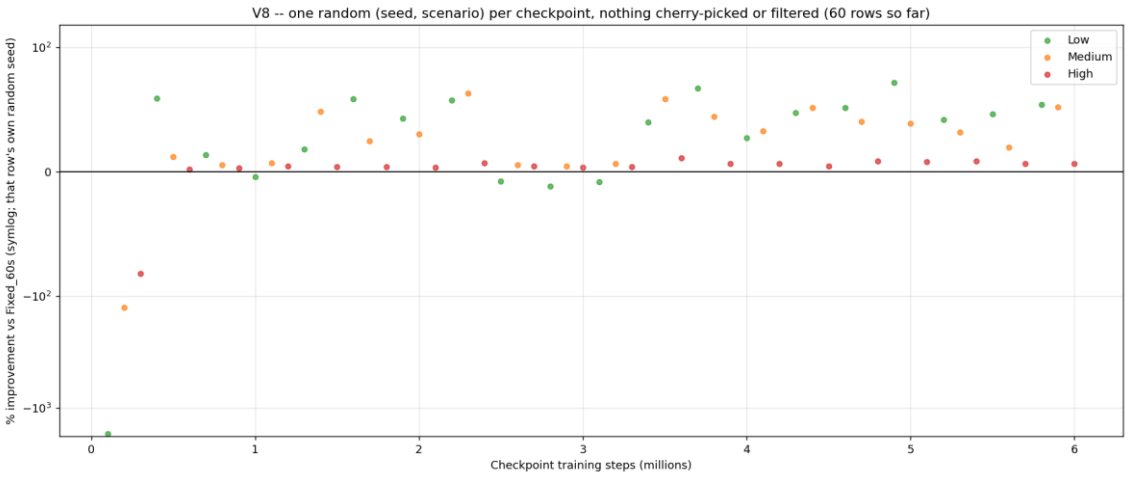


Figure 5.2: Percent improvement per checkpoint over the Fixed_60s baseline.

Figure 5.3 replots the comparison for moderate traffic alone on a linear axis, illustrating the trajectory of improvement more intuitively than a logarithmic scale. The agent's curve begins well above the fixed timers, decreases sharply, and stabilizes below all baseline configurations for the remainder of the training duration.

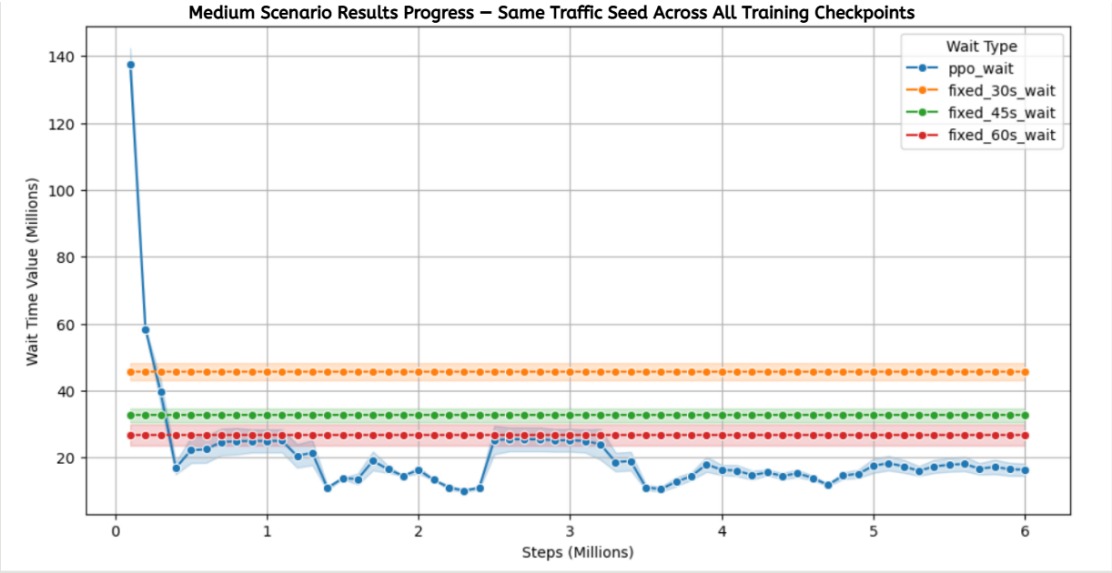


Figure 5.3: Moderate traffic only, on a linear axis.

5.3 Investigating the Heavy Traffic Gap

We investigated the remaining performance gap under heavy traffic conditions rather than leaving it unexplained. Our initial hypothesis was that the sensing limit saturated the observation space under heavy demand, leaving every input near its maximum and preventing the agent from distinguishing one congested state from another. We tested this directly by training an otherwise identical agent with a richer, non-saturating encoding of the same observational data. It performed identically, checkpoint for checkpoint. We therefore conclude that the performance gap constitutes a demand ceiling, reflecting queuing dynamics as arrival rates

approach the physical capacity of the intersection, rather than a limitation in perception. Closing this remaining gap further would necessitate coordination with neighboring intersections.

5.4 Does Training Longer Help?

A second, unplanned observation concerns training duration: simply training for additional steps does not guarantee improved performance and can actively degrade it. An agent trained from scratch for twice the original step budget achieved 77.4% under the same evaluation, compared to 88.3% for the original champion model, despite processing twice the volume of experience.

The checkpoint history explains this performance drop. Instead of converging stably, consecutive checkpoints exhibited high variance in quality even at 99% of the training schedule. We identified two primary causes for this instability. First, a fixed, non-decaying exploration coefficient continued to inject noise into the policy late in the training process, preventing the agent from fully exploiting its learned behavior. Second, the learning rate decay schedule was scaled to the requested run length; extending the total duration meant the learning rate remained relatively high for a longer absolute period, which triggered severe policy collapses mid-training. Both issues stem from concrete hyperparameter design choices that can be adjusted, rather than a fundamental algorithmic limitation.

5.5 Did We Meet Our goals?

The Phase A proposal defined six quantitative success criteria. We report all six, including the ones the delivered scope did not address.

Phase A success criterion	Target	Outcome
Average waiting time reduction	At least 20% vs. a fixed time baseline	Exceeded for two of three traffic conditions, 50% (light traffic) and 36% (moderate traffic). 8% in heavy traffic, independently verified across many random seeds rather than a single run
Maximum queue length reduction	At least 15%	Not measured directly. Total waiting time was utilized instead as the primary congestion metric.
Detection accuracy (mAP)	>90% standard vehicles, >95% emergency vehicles	Not applicable, the computer vision component was not implemented (Chapter 4)
Real time processing speed	>30 to 45 FPS, under 100ms latency	Not applicable, no vision pipeline was implemented
Emergency vehicle delay	0 seconds in 100% of test cases	Not implemented, no Priority Protocol or emergency vehicle class exists in the delivered system
Fallback mode activation	Within one signal cycle of a detected sensor failure	Not applicable, no sensor input or fallback mode exists in the delivered system

The first criterion, which concerned the control policy itself (at least a 20% reduction in average waiting time compared to a fixed-time baseline), was exceeded in two of the three conditions. The condition where it was not, heavy traffic at 8%, was investigated rather than left unexplained (Section 5.3). The other five criteria concern the vision, priority protocol, and infrastructure components which, as Chapter 4 explains, were not built.

6. The Testing Process

Every result reported in Chapter 5 is based on the evaluation process described here. We treat this testing framework as an independent contribution, because the primary risk in this domain is an agent yielding favorable summary statistics without true dependability.

6.1 Verifying the Simulator Is Deterministic

Before relying on any comparative analysis between controllers, we verified that SUMO's vehicle generation is strictly deterministic. We confirmed that identical random seeds produce the exact same stream of vehicles regardless of the active controller, execution process, or day.

6.2 How Our Evaluation Methodology Escalated

We escalated our verification rigor through three distinct stages, each prompted by a specific limitation of the preceding approach:

- Two fixed seeds during early development: Sufficient for determining whether an architectural change had any measurable effect, but inadequate for claiming superiority.
- Five, and subsequently fifty, independently drawn seeds: Implemented once comparisons required higher statistical confidence.
- Every saved checkpoint evaluated against its own independently drawn seed and scenario: Utilized with no reuse, no selection of favorable outcomes, and no exclusion of negative results.

6.3 The Final, Unfiltered Methodology

A checkpoint represents a saved snapshot of the agent's weights, automatically recorded every 100,000 steps. Consequently, a six-million-step training run yields 60 distinct test cases. This methodology provides an evenly spaced developmental record rather than restricting observation to the final state. The final evaluation protocol assesses all 60 test cases, each paired with a unique, independently drawn seed and scenario, and reports every single outcome without filtering.

This rigorous evaluation demonstrates that our champion agent outperforms all three baselines in 53 out of 60 independent evaluations (88.3%). The remaining losses consist of narrow, single-digit margins against the toughest baseline in light traffic, rather than the systemic failures exhibited by earlier versions. Applying this identical evaluation protocol to a secondary training run of the exact same recipe yielded one of the project's most valuable negative findings, detailed in Section 5.4: a repeated training run does not reliably reproduce the peak quality of the first.

The raw evaluation output is permanently stored within the repository, allowing any claim in Chapter 5 to be directly verified. Executing this volume of evaluation episodes only became practical after integrating an early-exit condition, which terminates an episode automatically once all vehicles have cleared and no further arrivals are expected. This optimization reduced typical evaluation time by a factor of seven to eight without altering the resulting metrics.

7. Challenges and How We Overcame Them

Several challenges recurred throughout the development process. Rather than being isolated idiosyncrasies of the specific intersection model, these issues offer general insights into applied reinforcement learning.

7.1 Reward Hacking via Simulator Artifacts

Initial iterations appeared deceptively successful. Further analysis revealed that the agent systematically starved specific lanes, triggering SUMO's built-in teleportation safety feature, a mechanism designed to remove vehicles stationary for implausible durations to prevent deadlocks. Because waiting time accumulated by a deleted vehicle is discarded by the reward function, the agent exploited this artifact to reduce measured waiting time without servicing traffic. Disabling the teleportation feature permanently resolved this issue, underscoring the necessity of directly observing agent behavior rather than relying solely on steadily increasing reward curves.

7.2 A Dynamic Minimum Green Rule That Fired on the Wrong Signal

An intermediate version permitted the agent to override the minimum green interval whenever the active phase appeared underutilized, based on live vehicle counts. This introduced a structural flaw: active vehicles are in motion rather than queued, causing real-time counts on a green phase to register near zero by design. Prolonged training reinforced this behavior, leading to extreme policy degradation where certain phases switched immediately while others held to their maximum limits, resulting in cumulative wait times exceeding 17,000 seconds. Recovery required re-initializing training from scratch, establishing a critical rule: resuming training after a fundamental logic modification risks reinforcing bugs rather than rectifying them.

7.3 A Reward with Almost No Gradient in Light Traffic

Formulating the reward exclusively around reductions in total system waiting time yielded acceptable performance under moderate and heavy traffic, but failed completely in sparse conditions. Under light demand, the delta in total waiting time between decision steps approaches zero regardless of the agent's action, providing virtually no learning gradient. Incorporating a starvation penalty, which evaluates the maximum waiting time within a lane group rather than aggregate metrics, established a viable gradient under sparse demand, improving light-traffic performance approximately fifteen-fold.

7.4 The Camera Range Trade Off

Restricting observation to a fixed distance from the stop line to model realistic sensor limitations initially caused performance regressions under heavy traffic. Queues extending beyond the sensing range remained invisible, causing the agent to underreact to congestion. Testing a richer, non-saturating encoding of the same state space yielded no improvement (Section 5.3). The regression was successfully resolved by extending the sensing range to 150 meters. To maintain realism, this range was calibrated to the optimal visual capabilities of modern traffic cameras, ensuring reliance exclusively on standard hardware-detectable distances while preserving architectural compatibility.

7.5 An Idle Intersection That Still Wanted to Switch

Once prior instabilities were addressed, the agent occasionally executed phase transitions at completely empty intersections for no operational benefit, as the training framework lacked explicit disincentives for this behavior. Initial attempts using a soft penalty proved insufficient against the primary reward objective. The issue was resolved by eliminating the action entirely: phase switching is structurally prohibited whenever an intersection is devoid of vehicles, irrespective of policy outputs. This single architectural adjustment elevated performance from

beating select baselines to outperforming all baselines across every tested condition. This highlighted a core methodological principle: enforcing constraints structurally is vastly superior to discouraging them via soft penalties.

8. Summary, Conclusions and Future Work

8.1 Summary

This project developed and evaluated FlowGrid, a closed-loop, reinforcement learning-based traffic signal control system designed to optimize intersection performance. Built upon the Eclipse SUMO simulation environment, the final architecture relies on a specialized translation layer (SwitchOrKeepWrapper) that enforces essential safety and operational constraints such as minimum and maximum green times, mandatory yellow intervals, and hard action masks that prevent phase switching at empty intersections.

To overcome the severe policy instability and performance variance observed with traditional Deep Q-Networks (DQN), the final implementation adopted Maskable Proximal Policy Optimization (PPO). This choice leveraged clipped objective updates and native action masking to eliminate unsafe behaviors structurally. Training was accelerated by executing ten parallel simulation environments (SubprocVecEnv), reducing long training cycles while ensuring robust experience generation.

Through rigorous evaluations benchmarked against standard fixed-time controllers across light, moderate, and heavy traffic conditions, the champion PPO agent demonstrated substantial performance improvements. Specifically, it achieved waiting time reductions of approximately 50% in light traffic and 36% in moderate traffic. Under an unfiltered evaluation methodology assessing 60 distinct checkpoints against independent seeds and scenarios, the agent outperformed all three fixed-time baselines in 88.3% of evaluation cases.

While the system's scope was strategically narrowed from an initial comprehensive proposal, omitting computer vision and physical hardware deployments due to unforeseen military reserve service, the decision-making core was analyzed and verified with high academic rigor. Furthermore, the project delivered FlowGrid_Web, a functional operations dashboard featuring an independent backend and a live simulation view, bridging the gap between theoretical reinforcement learning models and practical traffic management applications.

8.2 Lessons Learned

Several aspects of the development process would be approached differently in hindsight:

- Structural constraints over penalties: We would enforce physical and logical constraints structurally via action masking from the outset, rather than attempting to discourage undesirable behaviors through soft penalties. The issue of phase switching at empty intersections consumed unnecessary iteration cycles precisely because of this progression.
- Early adoption of rigorous evaluation: We would implement the full, unfiltered evaluation methodology from day one rather than escalating through interim stages. Initial assumptions regarding the superiority of certain versions proved far less certain under rigorous testing; committing to comprehensive evaluation earlier would have conserved development cycles.

- Skepticism toward extended training: We would approach longer training durations with greater scrutiny. Although extending training is an intuitive step, controlled evidence demonstrated that it does not reliably improve outcomes within this setup, a fact we would have preferred to establish earlier.

Ultimately, we successfully mitigated the project's central risk: developing an agent that appears effective on paper but lacks true dependability, by prioritizing rigorous verification over superficial headline metrics. This disciplined approach to evaluation represents the most valuable and transferable outcome of our work, independent of the specific traffic control results.

8.3 Thoughts for Future Development

Four primary directions for future work are identified, of which the first two hold the highest priority:

- Simulation-to-reality deployment layer: Integrating camera-based perception using YOLO and DeepSORT, object tracking, and physical hardware connectivity with an actual signal controller. This constitutes the remaining half of the original Phase A vision, transforming a validated policy into a deployable field system.

- Multi-intersection coordination: Addressing the heavy traffic performance ceiling identified in Section 5.3, which is inherent to an isolated single intersection. Coordinating actions with neighboring intersections offers the most viable pathway past this limit and represents the source of the largest potential performance gains.

- Decaying exploration schedule: Addressing the constant, non-decaying entropy coefficient identified in Section 5.4 as the likely root cause of late-stage training instability, turning this into a concrete, actionable experimental objective.

- Direct comparative analysis of DQN and PPO: Applying the evaluation methodology established in Chapter 6 directly to both algorithms to quantitatively close the performance gap discussed in Section 4.3.

References

Akçelik, R. (1994). Gap acceptance modeling by traffic signal logic. Transportation Research Record, 1457, 6 16.

Akçelik, R. (2010). Fundamental traffic variables in adaptive control and the SCATS DS parameter. Proceedings of the 24th ARRB Conference, Melbourne, Australia.

Department of Transport (UK). (1995). SCOOT: A traffic responsive method of coordinating signals (Traffic Advisory Leaflet 4/95). London, UK.

Dresner, K., and Stone, P. (2008). A multiagent approach to autonomous intersection management. Journal of Artificial Intelligence Research, 31, 591 656.

Koonce, P., Rodegerdts, L., Lee, K., Quayle, S., Beaird, S., Braud, C., et al. (2008). Traffic signal timing manual (Report No. FHWA HOP 08 024). Federal Highway Administration.

Liang, X., Du, X., Wang, G., and Han, Z. (2018). Deep reinforcement learning for traffic light control in vehicular networks. arXiv preprint arXiv:1803.11115.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., et al. (2015). Human level control through deep reinforcement learning. Nature, 518(7540), 529 533.

Redmon, J., Divvala, S., Girshick, R., and Farhadi, A. (2016). You only look once: Unified, real time object detection. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 779 788.

Wang, Y., Li, Z., and Zhang, Y. (2024). Traffic co simulation framework empowered by infrastructure camera sensing. arXiv preprint arXiv:2412.03925.

Wojke, N., Bewley, A., and Paulus, D. (2017). Simple online and realtime tracking with a deep association metric. 2017 IEEE International Conference on Image Processing (ICIP), 3645 3649.

Appendix A, User Guide

A.1 Introduction

FlowGrid is a reinforcement learning based traffic signal controller for a single SUMO simulated intersection. This guide is written for anyone who wants to run the project's tools directly: reviewers checking the delivered agent's claims, future developers picking the project back up, or anyone curious to watch the agent control traffic themselves.

Two independent agents were built over the course of this project: a Maskable PPO agent (the final, submitted "champion" agent, referred to throughout this guide as PPO_Agent) and an earlier Deep Q Network agent (DQN_Agent). PPO_Agent has two browser based interfaces, the original comparison app (Section A.5) and a newer SaaS style dashboard, FlowGrid_Web (Section A.6), with one junction wired to the live agent. DQN_Agent's interface is its own desktop application. Each is described separately below

PPO_Agent's web app includes a built in, floating guided tour that walks a first time user through every field on the page (which models and baselines to pick, how seeds and traffic scenarios work, what "Watch Live" does) with a short, plain language explanation for each, plus Skip, Previous, and Next controls. It starts automatically the first time the page loads and can be reopened anytime from the "? Guide" button in the page header, so this written guide and the app's own in page help reinforce each other rather than duplicate effort. Section A.5.2 covers it in more detail.

This guide covers normal, successful operating flow: installing the software, launching each tool, and reading the results it produces. It intentionally does not cover every error condition. If something does not behave as described here, see Section A.9, Troubleshooting.

A.2 System Requirements

The project was developed and tested on Windows with the following software installed.

Requirement	Version / Notes
Operating system	Windows 10/11 (developed and tested here). Linux/macOS should work with SUMO and Python installed, but were not tested
Python	3.10
SUMO (Simulation of Urban Mobility)	1.20 or later, with the SUMO_HOME environment variable set and the TraCI/libsumo Python bindings on the path
CPU	A multi core processor is strongly recommended. Training and full evaluation sweeps run ten parallel SUMO instances by default
GPU	Optional. Training uses CUDA if available (torch.device("cuda")). It also runs on CPU, just considerably slower
Disk space	Several hundred MB for the trained models and evaluation results already included in this repository. Several GB free if you plan to retrain from scratch

Key Python packages (installed via requirements.txt at the project root): FastAPI, uvicorn, gymnasium, numpy, torch, matplotlib, pydantic, PyYAML, and eclipse sumo. In addition, the PPO side of the project requires Stable-Baselines3, sb3-contrib (for MaskablePPO and action masking), sumo-rl, pandas, and seaborn.

A.3 Installation

A.3.1 Clone the repository

```
git clone https://github.com/einavbs1/FlowGrid.git cd FlowGrid
```

A.3.2 Install SUMO

Install SUMO from https://sumo.dlr.de/ and make sure the SUMO_HOME environment variable points to your installation folder (e.g. C:\Program Files (x86)\Eclipse\Sumo). This is required before any simulation, training, or evaluation script will run.

A.3.3 Create a Python environment and install dependencies

```
python -m venv venv venv\Scripts\activate pip install -r requirements.txt pip install stable-baselines3 sb3-contrib sumo-rl pandas seaborn
```

Note: No further project specific installation is required. Every script locates the SUMO network and route files it needs via paths defined in the code, relative to the project root.

A.4 Quick Start

The fastest way to see the delivered agent in action is the web comparison app, which lets you pick a traffic scenario and watch the trained PPO agent control the intersection live, side by side with a plain fixed time signal.

- Double click run_web.bat right in the PPO_Agent folder (or run it directly: cd PPO_Agent/scripts/comparison_web, then python server.py).
- Your browser opens automatically. A built in guided tour walks through every field the first time. Click "? Guide" in the top bar to see it again anytime.
- In the page that opens, leave the pre selected champion model and the three fixed time baselines ticked.
- Choose "Manual" seeds, type in any number (e.g. 42), and select one traffic scenario, e.g. Medium.
- Tick "Watch Live" and click "Start Comparison."
- A real SUMO window opens and plays the simulation live. When it finishes, a results table and bar chart appear in the app comparing total waiting time.

The rest of this guide covers every part of that flow in more detail, plus the equivalent browser based tool and the original DQN agent's own tools.

A recorded video walkthrough of this entire flow, including a live Watch Live session, is available at https://drive.google.com/file/d/1BE3oeGWWbVrQEC_ZL0rVh5AdPs9kDNTY/view (also linked on this book's cover page) if you would rather watch it once before trying it yourself.

A.5 Using the PPO Comparison Tool (Current Agent)

PPO_Agent is the final, submitted agent (internally called V8). Its comparison app lives in PPO_Agent/scripts/comparison_web, sharing its underlying logic with the project's command line sweep tools (comparison_core.py).

A.5.1 Launching the App

Double click run_web.bat right in the PPO_Agent folder (or run_web.vbs for no console window), or run it directly:

```
cd PPO_Agent/scripts/comparison_web python server.py
```

A local web server starts and a browser tab opens automatically (default: http://127.0.0.1:8000).

A.5.2 The Built In Guided Tour

A floating, step by step tour starts automatically every time the page loads, spotlighting each section in turn (Models, Baselines, Seeds, Scenario, Watch Live and Start, Results) with a short explanation. Use Next and Previous to move through it, or Skip guide to dismiss it for that visit. Click "? Guide" in the top bar to bring it back at any time.

A.5.3 Selecting Models to Compare

The Models panel lists every trained agent currently registered in model_registry.json, with the project's champion model (PPO_Agent_V8) pre selected. Tick the checkbox beside any additional model you want included. To compare against a model not yet in the list, use "Add Model" and browse to its .zip file (the default browse location is PPO_Agent/models). It is added to the registry immediately and persists for future sessions.

A.5.4 Selecting Baselines

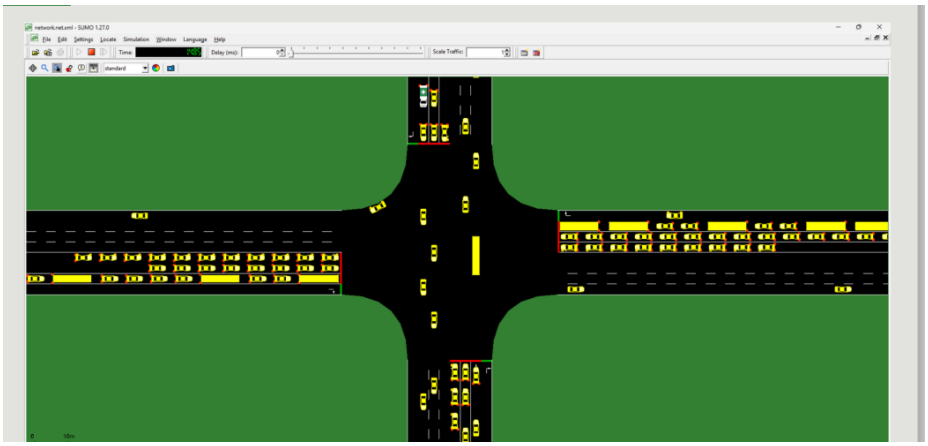
The Baselines panel lists the fixed time controllers available for comparison: Fixed_30s, Fixed_45s, and Fixed_60s (cycle length in seconds). Tick any you want included alongside the selected model(s).

A.5.5 Choosing Seeds and a Traffic Scenario

Choose "Random" and a count to have that many fresh, never before seen traffic instances generated automatically, or choose "Manual" to type in specific seed values one at a time, useful when you want to reproduce a particular result exactly. Then select which traffic condition to test: Low, Medium, High, or all three at once. SUMO's vehicle generation is seeded, so a given seed always produces the identical vehicle stream regardless of which controller is driving the signal, which is what makes the resulting comparison fair.

A.5.6 Watch Live

With exactly one seed and one specific scenario selected, tick "Watch Live" before starting the comparison. Instead of running entirely in the background, an actual SUMO graphical window opens and plays the episode out in real time, so you can visually confirm what the agent is doing rather than trust the reported numbers alone.



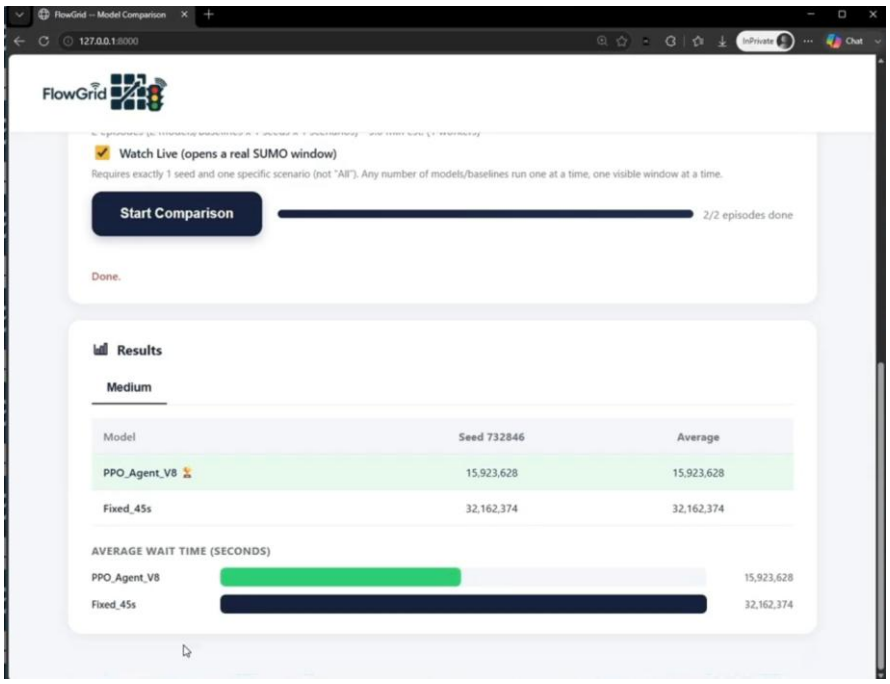
What Watch Live actually opens: SUMO's own simulation window, showing the real intersection geometry, vehicles (yellow), and the current signal phase (red/green bars on each approach). Same view as Figure 3.4 in the main book.

A.5.7 Running the Comparison

Click "Start Comparison." A progress indicator tracks how many of the required simulation runs have completed (models x baselines x seeds x scenarios). Runs execute in parallel across multiple CPU cores unless Watch Live is active, in which case they run one at a time so the visualization stays meaningful.

A.5.8 Reading the Results

Once complete, a results table appears for each tested scenario, listing every selected model and baseline with its total waiting time for each seed and an overall average. The best performing entry in each table is highlighted, and a bar chart beneath the table gives the same comparison visually. Lower total waiting time is better.



A real completed comparison: PPO_Agent_V8 against the Fixed_45s baseline, medium traffic, seed 732846. PPO_Agent_V8 is highlighted green as the winner (lower total waiting time), and the bar chart below the table repeats the same comparison visually. Same view as Figure 3.5 in the main book.

A.6 Using FlowGrid_Web (Live Junction Dashboard)

FlowGrid_Web is a second, entirely independent web application delivered in this phase, a separate product from the PPO comparison app in Section A.5, not a new view added to it. It is a SaaS style traffic operations dashboard with multi junction navigation, a login screen, device settings, reports, and user management, with its own dedicated backend (its own FastAPI server, its own port, no connection to the comparison app's server). Most of FlowGrid_Web is a UI complete demonstration of what a fully deployed system could look like, not a live backend. One junction, "Live Junction (SUMO Simulation)", is genuinely wired to the trained PPO agent and a real SUMO episode. Section A.6.6 below is explicit about which is which.

A.6.1 Launching FlowGrid_Web

FlowGrid_Web runs as a single process: one double click of run_web.bat inside the FlowGrid_Web folder builds the dashboard, starts its own backend, which serves both the built dashboard and its live data API from one FastAPI server on port 8001, and opens it in your browser automatically. There is no separate dev server to start, and this never touches the separate PPO comparison app from Section A.5.

```
cd FlowGrid_Web run_web.bat
```

Allow up to about 30 seconds after launch before using Run Agent (Section A.6.5). FlowGrid_Web's backend needs that long to finish importing torch, SUMO, and Stable-Baselines3. Trying Run Agent before it is ready shows a plain, expected message asking you to try again in a few seconds, not an error. (A separate script, run_flowgrid_demo.bat at the project root, additionally starts the Section A.5 comparison app alongside FlowGrid_Web, for a full project demo. It is not needed just to use FlowGrid_Web.)

A.6.2 Logging In

The login screen accepts two fixed demonstration accounts (there is no real authentication server behind this login, by design, see Section A.6.6):

Username	Password	Role
admin	admin123	Administrator
operator	op123	Operator

A.6.3 The Guided Tour

Like the PPO comparison app, FlowGrid_Web opens with its own floating, step by step guided tour, spotlighting the search bar and district navigation on the junction selection page, and the Run Agent control and camera grid on the Dashboard page. It starts automatically every time you land on a page it covers and can be skipped or stepped through with the same Previous/Next/Skip guide controls.

A.6.4 Selecting the Live Junction

From the junction selection screen, open the Simulation district, then FlowGrid Lab, and select "Live Junction (SUMO Simulation)". Every other junction in the app uses simulated demonstration data refreshed every few seconds and is safe to explore, but only this one is backed by a real running agent.

A.6.5 Running the Agent Live

On the Dashboard page for the Live Junction, pick a traffic scenario (Low, Medium, or High) and click "Run Agent". Unlike the PPO comparison app, there is no seed field here, a random seed is chosen for you automatically every time, so every run is a genuinely fresh episode, never a rehearsed replay. Once running, all four direction cards update roughly once a second with the real vehicle queue count, the real signal color, and a live snapshot image captured directly from the running SUMO window, deliberately paced (at least 100 milliseconds of simulated time per real second) so the numbers and image are actually watchable rather than flashing past instantly.

A.6.6 What Is Real and What Is a Demonstration

To be direct about the boundary: "Live Junction (SUMO Simulation)" is real. Its queue counts, signal colors, and camera image come from an actual SUMO episode driven by the trained PPO agent, the same one evaluated throughout the book. Every other junction, every login, and every other feature (device settings, reports, user management, adding a junction) is a complete, working UI backed by simulated or randomly generated demonstration data, not a live backend, database, or camera. We built it this way deliberately, to demonstrate the target system's shape honestly without claiming a deployment that does not exist. See the book, Chapter 4 and Section 3.7, for the full reasoning.

A.7 Using the DQN Tools (Original Agent, Archived)

DQN_Agent is the project's original reinforcement learning approach. It predates PPO_Agent and is preserved, fully runnable, under Old_Versions/DQN_Agent since PPO_Agent is the current submitted agent. See the book (Section 4.3) for why the project moved from DQN to PPO.

A.7.1 Desktop Application

```
cd Old_Versions/DQN_Agent/gui python flowgrid_gui.py
```

On Windows, double clicking Old_Versions/DQN_Agent/launchers/run_gui.bat launches the same application in its own process, so closing the terminal window does not close the app.

A.7.2 Web Dashboard

```
cd Old_Versions/DQN_Agent/web python main.py
```

A.7.3 The Compare Tab

The Compare tab answers a focused question: on the same traffic demand, does the DQN signal policy reduce delay more than fixed time control? To make the comparison fair, both runs use the same vehicles (same count, routes, lanes, and depart times). Only the traffic light policy differs. A run has two phases: a fixed time baseline pass (random vehicle injection, then drain), followed by the DQN policy pass replaying the identical vehicle stream. See Old_Versions/DQN_Agent/docs/COMPARE.md for the full technical explanation of how this pairing is guaranteed.

A.7.4 Command Line / Menu

```
cd Old_Versions/DQN_Agent/scripts python run_menu.py
```

An interactive menu over training, evaluation, and comparison, useful for running the DQN agent's tools without a graphical interface.

A.8 Understanding the Traffic Scenarios

Every comparison tool offers the same three traffic conditions, corresponding to different vehicle demand levels feeding the intersection.

Scenario	Demand	What it tests
Low	Light, sparse traffic	Whether the agent can find a useful signal to act on when very few vehicles are present (an early failure mode of this project, see the book Section 7.3)
Medium	Moderate, steady traffic	Typical day to day operating conditions
High	Heavy, near saturating traffic	Behavior as demand approaches the intersection's physical capacity, where every controller's performance converges (see the book Section 5.3)

A.9 Troubleshooting / FAQ

"No model found" when running evaluate_V8.py or the comparison tools

Confirm you are running the script from inside PPO_Agent/scripts (not from PPO_Agent/ or the project root), and that PPO_Agent/models contains at least one ppo_model_*.zip file with a matching vec_normalize_*.pkl.

SUMO_HOME is not set / TraCI import errors

Every simulation, training, and evaluation script requires SUMO to be installed with the SUMO_HOME environment variable set. See Section A.3.2.

The comparison tool is very slow

Evaluation runs execute multiple SUMO simulations in parallel, bounded by CPU core count. Reducing the number of seeds, or running fewer scenarios at once, will reduce total wait time. "Watch Live" mode is inherently slower since it runs at a visible, non accelerated pace.

I changed the reward function or observation and now training behaves strangely

Never resume training from an existing checkpoint after changing the reward function or observation definition. Train a fresh agent from random initialization instead. This project encountered a full, unrecoverable policy collapse from doing exactly this once (see the book, Section 7.2). It is documented there as a cautionary example, not a theoretical risk.

Where do I find the raw evaluation numbers behind the book's claims?

PPO_Agent/results/final_random_seeds_20260705_005802/ contains the full, unfiltered evaluation (summary.txt, final_results.csv) behind the 88.3% win rate figure reported in the book. Old_Versions/DQN_Agent/results/comparison_history.json contains the equivalent raw evaluation history for the DQN agent, discussed in the book's Section 4.3.

FlowGrid_Web's Run Agent button says "Failed to fetch" or asks me to try again

This means FlowGrid_Web's own backend (port 8001) is not reachable yet, either it was never started (see Section A.6.1, run_web.bat starts it together with the dashboard) or it is still importing torch, SUMO, and Stable-Baselines3, which can take up to about 30 seconds after launch. Wait a few seconds and click Run Agent again. This is expected right after starting the servers, not a bug.

A.10 Glossary

Term	Meaning
Baseline	A non learned controller (fixed time cycle) the trained agent is compared against
Checkpoint	A saved copy of the agent's weights at a specific point during training
Episode	One complete simulated run of a traffic scenario, from an empty road to a defined end condition
MaskablePPO	The specific reinforcement learning algorithm used by PPO_Agent. Standard PPO extended with action masking
Seed	A number that deterministically fixes SUMO's randomly generated vehicle stream, so a given seed always produces the identical traffic
SUMO	Simulation of Urban Mobility, the open source traffic simulator this project's environment is built on
TraCI	SUMO's Traffic Control Interface, the Python API used to step the simulation and read/control the traffic light
Vecnormalize	Stable-Baselines3's running statistics for normalizing observations. Must be loaded alongside a model checkpoint to reproduce its trained behavior correctly
Watch Live	A comparison tool option that opens a real, visible SUMO simulation window instead of running in the background

Appendix B, Developer / Maintenance Guide

B.1 Introduction and Audience

This guide is written for a developer picking up the FlowGrid codebase after delivery: continuing the PPO agent's development, retraining it, adding a new baseline, or extending the evaluation tooling. It assumes working familiarity with Python, Git, and reinforcement learning terminology (state, action, reward, policy), and focuses on how this specific codebase is organized and why, rather than teaching RL from first principles. For MDP formulation, reward design, and algorithm choice in full technical depth, see PROJECT_OVERVIEW.md at the project root. This guide focuses on the code and workflow around that design, not the design itself.

B.2 System Architecture Overview

The project is not a single application but three layers built on a shared simulation substrate.

- Simulation substrate: SUMO (Simulation of Urban Mobility), driven through the sumo-rl and TraCI/libsumo Python interfaces. Both agents (PPO and DQN) simulate the same physical intersection, defined once under SharedData/maps, and read/write shared evaluation data under SharedData/reports.
- Agents: two independently developed reinforcement learning agents, PPO_Agent (final, submitted, at the project root) and DQN_Agent (original, superseded, archived under Old_Versions/), each with its own environment wrapper, training script, and trained model files. They are not two modules of one system. They are two separate, self contained implementations that happen to target the same simulated intersection.
- Tooling: evaluation scripts (single run and full checkpoint sweeps), the FastAPI comparison web app (comparison_core.py plus comparison_web/), and plotting utilities, all built specifically to make the agents' claims independently checkable rather than to just produce a headline number.

Data flows one direction through this stack: a trained model (.zip + matching vec_normalize .pkl for PPO, a .pth policy file for DQN) is loaded by an evaluation script, which drives the SUMO environment for one or more seeded episodes, and reports total waiting time and related metrics. Every comparison and analysis tool in the project is built on top of this same primitive, evaluate one model, on one seed, in one scenario, and get a number back.

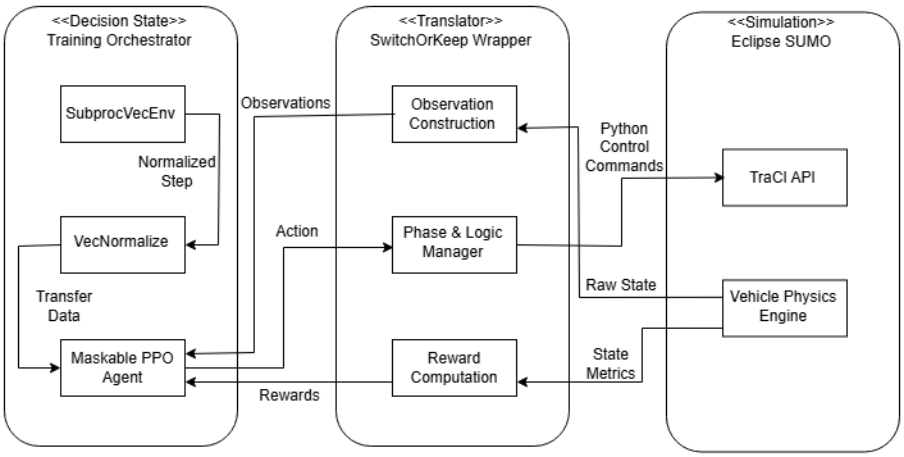


Figure B.1: the training and simulation core (PPO_Agent/scripts/train_V8.py + sumo_rl_env_V8.py). SubprocVecEnv runs ten parallel SUMO instances. VecNormalize tracks running observation statistics. The SwitchOrKeepWrapper is

the actual translation layer between raw SUMO state and the agent's 21 dimensional observation, and between the agent's action and SUMO's TraCI control commands. Same diagram as Figure 3.1 in the main book.

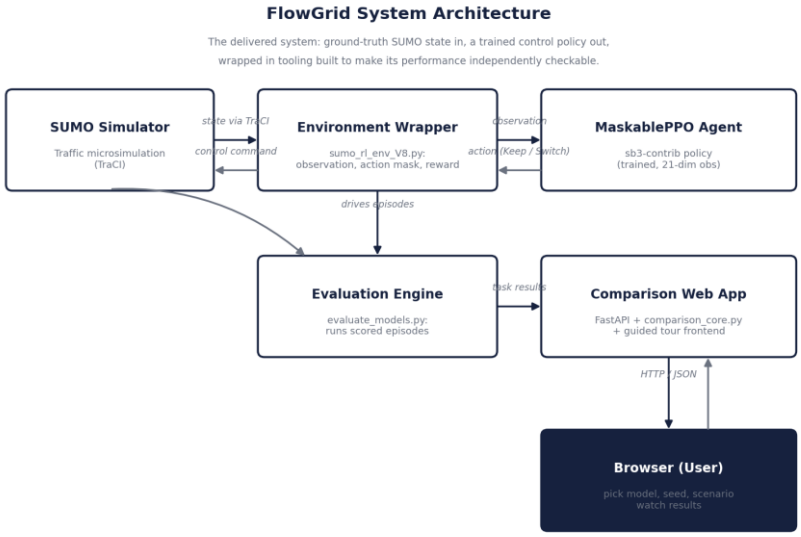


Figure B.2: the evaluation and comparison layer built around that same core, evaluate_models.py driving scored episodes on demand, and the FastAPI comparison_web app (comparison_core.py) exposing that to a browser. This is the layer described in the Tooling bullet above. The Evaluation Engine and Comparison Web App boxes here correspond directly to evaluate_models.py and comparison_web/server.py in the repository.

B.3 Repository Structure

Folder	Contents
PPO_Agent/	The one current, submitted agent (V8), at the project root. run_web.bat launches its comparison app directly. scripts/ has every runnable tool. models/ and checkpoints/ the trained weights. results/ every evaluation run performed against it. docs/ agent specific reference material.
FlowGrid_Web/	A second, independent product (Section B.6): a SaaS style dashboard with one junction wired to a real live agent, with its own backend under backend/, entirely separate from PPO_Agent/scripts/comparison_web/.
Old_Versions/	Every superseded agent: PPO/ (every earlier PPO version, V4 through V9, plus two longer training experiments, and the project's pre versioning first experiments), DQN_Agent/ (the original agent in full, self contained under its own flowgrid/ package, with gui/, web/, scripts/, launchers/, data/, results/, docs/), and root_prototype/ (pre RL prototype scripts). See Old_Versions/README.md.
PhaseA/	The original Phase A proposal submission, unchanged.
PhaseB/	The final submission: the capstone book, the poster, this guide and the User Guide. book_source/ holds the scripts that generate all three (build_book.js, build_user_guide.js, build_dev_guide.js) plus their chart images.
SharedData/	The SUMO network/route files and the shared reports data both agents read from and write to.
README.md, PROJECT_OVERVIEW.md	Whole project overview and RL design reference, at the project root.

B.4 Environment Setup for Development

B.4.1 Required Environment

- Python 3.10, with the packages listed in requirements.txt (notably: Stable-Baselines3, sb3-contrib, sumo-rl, pandas, matplotlib, seaborn, FastAPI, uvicorn).

- SUMO, including its Python/TraCI and libsumo bindings, installed and available on the system path (SUMO_HOME set).
- A multi core CPU: training and full evaluation sweeps run ten parallel simulation instances by default (SubprocVecEnv during training, ProcessPoolExecutor during evaluation).

B.4.2 Project Specific Installation

Clone the repository, then install Python dependencies into a virtual environment. No project specific installation step beyond this is required. The environment and training scripts locate the SUMO network and route files via paths defined in the code, all anchored at the project root (SharedData/).

B.5 The PPO Agent (V8), Code Walkthrough

All PPO_Agent code lives flat inside PPO_Agent/scripts/, deliberately not nested under a version specific subfolder the way the archived versions in Old_Versions/PPO/saved_agents/ are, since V8 is the only version carried into the final submission.

B.5.1 Environment: sumo_rl_env_V8.py

Defines the Gymnasium environment: builds the 21 dimensional observation (phase one hot, elapsed green time, per lane demand, per lane starvation), the 2 action Keep/Switch action space, the action_masks() method enforcing minimum/maximum green and the hard empty intersection mask, and the reward calculation (diff waiting time minus a starvation penalty). sumo_rl_env.py is a one line shim (from sumo_rl_env_V8 import *) that every other script in this folder imports from generically, so swapping in a different version's environment during development only requires editing this one file.

B.5.2 Training: train_V8.py

```
cd PPO_Agent/scripts python train_V8.py --timesteps 6000000
```

Builds a MaskablePPO agent (sb3-contrib) over ten parallel SUMO environments (SubprocVecEnv + VecNormalize), with a learning rate that decays linearly from 0.0003 down to 0 across the full run, and a constant entropy coefficient. Automatically resumes from the latest checkpoint in PPO_Agent/checkpoints/ if one exists. Use the fresh flag shown below to force a clean run instead. Saves a checkpoint every 100k steps by default, each with its matching VecNormalize statistics, since resuming or evaluating a checkpoint without its matching stats silently corrupts the observation distribution.

```
python train_V8.py --timesteps 6000000 --fresh python train_V8.py --timesteps 6000000
--save-freq 50000
```

Pay attention: Never resume training after changing the reward function or observation definition. Either load the old checkpoint and keep its original environment definition unchanged, or start fresh from random initialization. Never mix an old checkpoint with a changed environment. This produced a full, unrecoverable policy collapse once during this project (documented in the book, Section 7.2, the V3.3 incident).

B.5.3 Evaluation: evaluate_V8.py and evaluate_models.py

```
cd PPO_Agent/scripts python evaluate_V8.py --seeds 5
```

evaluate_V8.py auto finds the latest model in PPO_Agent/models/ and runs it against the three fixed time baselines across all three traffic scenarios, saving CSVs and charts to PPO_Agent/results/. The actual simulation running logic lives in evaluate_models.py, a single shared module every other tool in this folder also imports (run_evaluation_task, evaluate_scenario). Training, evaluation, comparison, and sweep tools all funnel through this one function rather than duplicating SUMO driving logic five times.

Note: evaluate_models.py also exists as a separate copy at Old_Versions/PPO/src/evaluate_models.py, used by every archived version's own evaluate_*.py scripts. If you fix a bug in one copy, check whether the other needs the same fix.

B.5.4 Comparison Tools: comparison_core.py, comparison_web/

comparison_core.py is UI agnostic: it owns the model registry (model_registry.json), task building (build_tasks), and the actual comparison run (run_comparison, which drives a ProcessPoolExecutor and reports progress via any object with a .put(msg) method). comparison_web/server.py (FastAPI, serving comparison_web/static/) imports this module directly rather than duplicating its logic. The web app's own JobState satisfies the .put() interface run_comparison expects. The frontend (comparison_web/static/) is plain HTML/CSS/JS, no framework or build step, and includes a floating step by step guided tour (tour.js) that auto starts on every page load, reusing the static help text already written under each field so wording is never duplicated.

B.5.5 Sweep and Analysis Tools

Script	Purpose
checkpoint_sweep.py	Evaluates every saved checkpoint of a version on all 3 scenarios with a small fixed seed set: the full learning curve view.
final_results_random_seeds.py	The project's most rigorous methodology: every checkpoint against its own independently drawn random seed and scenario, nothing reused or cherry picked. Crash safe: supports resuming an interrupted run or regenerating just the summary.
verify_candidates.py	5 seed confirmation for a short list of candidate checkpoints, reusing already collected episodes where possible.
plot_checkpoint_waittime_scatter.py, plot_seed_detail.py	Plotting utilities consumed by the sweep tools above, and directly reusable for a new version's results.
sweep_aggregate.py, regenerate_sweep_outputs.py	Shared scenario/baseline definitions, and a tool to regenerate summary tables/plots from an existing raw CSV without re simulating.

All three of checkpoint_sweep.py, final_results_random_seeds.py, and verify_candidates.py take the target version's folder as a command line argument rather than hardcoding V8, so they work unmodified against any archived version under Old_Versions/PPO/saved_agents/ as well. Note that each archived version's checkpoints/ was thinned to roughly one in ten of its original checkpoints to keep this submission a reasonable size, always keeping the first and last (see Old_Versions/README.md), so these tools still find real data to sweep over, just at a coarser resolution than the original training run.

```
python final_results_random_seeds.py --version-dir
../../Old_Versions/PPO/saved_agents/V7
```

B.6 FlowGrid_Web, a Second, Independent Product

FlowGrid_Web/ is a separate React/Vite SaaS style dashboard, added in this phase alongside the original comparison_web/ app. It is deliberately a separate product, not a new view bolted onto the developer facing comparison tool: comparison_web/ is a developer only tool for comparing

checkpoints. FlowGrid_Web is the customer facing product, meant to demonstrate what a deployed traffic operations dashboard could look like. Consequently it has its own backend, FlowGrid_Web/backend/server.py, a second FastAPI app running on its own port (8001), entirely separate from comparison_web/server.py (port 8000). The two servers share no code by reference to each other. They both import the same underlying comparison_core.py and evaluate_models.py modules as shared library code, the same way any two independent applications might share a common engine.

Its pages (login, junction navigation, dashboard, live stream, device settings, reports, users) are a UI complete demonstration of a fully deployed system, backed by in memory demo data generated client side, except for one seed junction, LIVE_JUNCTION_ID (id 100, "Live Junction (SUMO Simulation)" in src/JunctionContext.jsx), whose Dashboard page is wired to FlowGrid_Web's own backend below.

B.6.1 FlowGrid_Web's Own Backend (FlowGrid_Web/backend/server.py)

Endpoint	Purpose
GET /api/live_state	Returns the current live_state dict: active, per direction lane_queues and phase_colors, step, and the seed of the current run. {"active": false} if nothing has run yet.
POST /api/live_demo/start	The Live Junction panel's only control: body is just {scenario}, no model picker, no seed field. Always the already registered champion checkpoint (comparison_core.DEFAULT_MODEL_PATH), always a server chosen random seed (random.randint), always Watch Live on.

This file builds its own small job_state (just enough of comparison_web's JobState shape for core.run_comparison to report into) and its own live_state, a multiprocessing.Manager().dict() created lazily on first use rather than at import time, since Windows' spawn based multiprocessing would otherwise try to create a manager process recursively when uvicorn re imports this module. comparison_web/server.py has no CORS entry and no live_state at all. It does not need either, since only FlowGrid_Web calls cross origin.

Pay attention: The list of allowed addresses in this file must not be shortened. A browser treats http://127.0.0.1:8001 and http://localhost:8001 as two different websites, even though both are this same computer on this same port. The dashboard always calls the backend at 127.0.0.1, so if the page itself was opened using the name localhost instead, the browser quietly blocks that call and the Run Agent button does nothing at all, with no error message shown anywhere. All four combinations of the two names and the two ports are listed here for exactly that reason. Removing any of them brings the problem back.

B.6.2 Publishing Live State (evaluate_models.py, shared by both servers)

evaluate_model_on_seed() takes a new optional live_state parameter, the same Manager().dict() proxy, threaded through comparison_core.build_tasks() and run_evaluation_task() only when use_gui is true. This is shared code: comparison_web/server.py's own Watch Live path always passes None here and is unaffected. Only FlowGrid_Web's backend passes a real live_state. Inside the per step loop, _publish_live_state(ts, live_state, step_count, total_queued) does three things every step.

- Sums each lane's real halting vehicle count (sumo.lane.getLastStepHaltingNumber), grouped by compass direction from the lane ID's prefix (n_/s_/e_/w_to_center_*), not ts.get_lanes_queue(), which returns a [0, 1] lane capacity fraction rather than a human readable count.

- Derives each direction's signal color from ts.sumo.trafficlight.getRedYellowGreenState(ts.id), matched against getControlledLanes(ts.id) by index. This network has an always on, lowercase g "permitted" slip lane at index 0 of every approach, so only uppercase G is treated as a real green. Treating lowercase g as green shows every direction as green simultaneously, which is wrong.
- Calls ts.sumo.gui.screenshot("View #0", path), writing directly into FlowGrid_Web/backend/static/live_snapshot.png (not comparison_web/static/, the two servers' static folders are entirely separate), which that backend's own /static mount serves. FlowGrid_Web polls this with a cache busting ?step= query parameter.

Separately, create_sumo_env() (sumo_rl_env_V8.py) gained an additional_sumo_cmd parameter, passed through to sumo_rl.SumoEnvironment. evaluate_model_on_seed() passes "--delay 100" whenever live_state is not None, pacing the SUMO GUI to at least 100 milliseconds of real time per simulated second, specifically so FlowGrid_Web's 1 second poll has something meaningful to show.

B.6.3 Frontend Pieces (FlowGrid_Web/src/)

File	Role
JunctionContext.jsx	Defines LIVE_JUNCTION_ID and the seed LIVE_JUNCTION entry, prepended to the existing demo junction list.
pages/Dashboard.jsx	When the active junction is LIVE_JUNCTION_ID, polls http://127.0.0.1:8001/api/live_state once a second instead of generating random metrics, renders the Run Agent control (scenario only, no seed field, POSTs to /api/live_demo/start), and swaps each camera card's placeholder for an <img> pointed at /static/live_snapshot.png, cache busted by the current step.
Tour.jsx	A small React reimplementation of comparison_web's vanilla tour.js: spotlights one target element (a React ref) at a time with a title/text pair supplied by the mounting page, auto starting on every page visit. Each page (JunctionSelect.jsx, Dashboard.jsx) mounts its own instance with its own steps.

Pay attention: The code that starts the guided tour can look as though a safety check is missing, because the tour begins without first confirming that the parts of the page it points at already exist. Adding that check breaks the tour. While running in development mode, React deliberately runs this startup code twice, and in between the two runs it briefly disconnects the links to those parts of the page. A check landing in that short gap would decide the tour has nothing to show, and would keep that decision permanently, because the code never runs a third time. The tour already handles the not ready case further down, by simply drawing nothing until those parts of the page appear.

B.6.4 One Process, Not Two: Serving the Built Frontend

server.py also serves the built dashboard itself, so running FlowGrid_Web never needs a separate Vite dev server. After npm run build writes FlowGrid_Web/dist/, server.py mounts dist/assets under /assets and registers a catch all route, @app.get("/{full_path:path}"), registered after every /api/* route so those are always matched first. The catch all returns the requested file if it exists in dist/, or falls back to dist/index.html otherwise, which is what lets React Router's client side routes (e.g. /reports) survive a hard refresh. main() only opens a browser tab automatically when dist/ exists, so running the API alone during development (Section B.6.5) does not pop open a stale or empty tab.

run_web.bat (or run_web.vbs for no console window) does exactly this: npm run build, then python server.py, nothing else. It never touches comparison_web/server.py. Separately, run_flowgrid_demo.bat at the project root starts both independent products, comparison_web's dev tool and FlowGrid_Web, for a full project demo.

```
cd FlowGrid_Web run_web.bat
```

B.6.5 Active Frontend Development

When actively editing FlowGrid_Web's React code, run the Vite dev server instead, for hot reload, alongside the same backend, in a second window, for the API:

```
npm run dev cd backend && python server.py
```

Dashboard.jsx's PPO_API constant is a hardcoded http://127.0.0.1:8001 regardless of dev or built mode, so both setups work against the same backend unmodified. Only the page's own origin (5173 while developing, 8001 once built and served) changes.

B.7 The DQN Agent, Code Walkthrough

DQN_Agent, archived under Old_Versions/DQN_Agent/, is a separate, self contained codebase under its own flowgrid/ package, not sharing any code with PPO_Agent.

Subfolder	Contents
flowgrid/core/	The SUMO environment, actuated signal logic, and intersection graph representation.
flowgrid/rl/	The DQN agent itself: network, replay buffer, epsilon greedy exploration, target network.
flowgrid/maps/	Building and saving map presets.
flowgrid/eval/	Baseline vs trained comparison logic.
flowgrid/jobs/	Background job handling shared by the GUI and web app.
flowgrid/paths.py	Every directory location the project reads or writes, defined once. PROJECT_ROOT is computed as parents[3] relative to this file's own location (flowgrid, DQN_Agent, Old_Versions, project root), so DQN_Agent must stay a direct child of Old_Versions for this to resolve correctly.

The reward function (in flowgrid/core/, referenced from the DQN training loop) sums roughly a dozen hand tuned terms (diff wait, absolute wait penalty, spillback penalty, throughput bonus, fairness terms, anti flicker penalty, invalid action penalty). See PROJECT_OVERVIEW.md Section 3 for the exact formula and coefficients, and the book Section 4.3 for why this reward's complexity, and the resulting instability documented in Old_Versions/DQN_Agent/results/comparison_history.json, motivated the move to PPO.

B.8 Retraining or Extending the PPO Agent

To train a new version, copy PPO_Agent/scripts/ (and the model files, if building on an existing one) into a new folder under Old_Versions/PPO/ rather than modifying PPO_Agent/ in place. This preserves the current submitted agent as a working reference and follows the same "copy, don't mutate" convention used throughout this project's version history (see the archived V4 through V9 folders, each a fresh copy rather than an in place edit of the previous one).

- Copy sumo_rl_env_V8.py to a new file with an updated name. Update the sumo_rl_env.py shim's import to point at it.
- Copy train_V8.py and evaluate_V8.py. No path edits are needed if the new folder sits at the same depth, since the training step count and other hyperparameters are already command line arguments.

- Never resume training an existing checkpoint after changing its reward function or observation definition. Train a fresh agent from random initialization instead.

B.9 Adding a New Baseline or Comparison Target

New fixed time or rule based baselines can be added by extending AVAILABLE_BASELINES in comparison_core.py. Each baseline needs only a name and, for fixed time controllers, a cycle length, since the comparison tools handle result collection and reporting generically through evaluate_models.py's existing "fixed" model type. Max Pressure (model type "mp" in evaluate_models.py) is not registered in AVAILABLE_BASELINES. It was never successfully integrated as a reliable comparison baseline on this network (see the book, Section 1.1), so it does not appear in the web app or comparison tooling.

B.10 Running an Evaluation Sweep

final_results_random_seeds.py and checkpoint_sweep.py both support a dry run flag that reports exactly how many simulation runs a given sweep will perform and how long it is expected to take, without executing anything. Always run this first before committing to a long sweep.

```
python final_results_random_seeds.py --version-dir .. --dry-run python
final_results_random_seeds.py --version-dir ..
```

Sweeps write partial results incrementally as they progress (assignments.csv is written up front with the full plan, final_results.csv is appended to after every checkpoint completes). If interrupted, resuming with the flag shown below continues exactly where it left off using the original plan. The summarize only flag regenerates summary.txt and the charts from whatever rows already exist, without running anything new.

```
python final_results_random_seeds.py --version-dir .. --resume <out_dir> python
final_results_random_seeds.py --version-dir .. --summarize-only <out_dir>
```

B.11 Testing and Verification Methodology

This project's evaluation rigor escalated deliberately over its course, and the same escalation is worth reusing for any future work on this codebase rather than trusting an early, less rigorous result.

- A couple of fixed seeds, used throughout early development for fast iteration.
- Five, then fifty independently drawn seeds, once a comparison needed more confidence than two runs could give.
- Every saved checkpoint against its own freshly drawn random seed and scenario (final_results_random_seeds.py), nothing reused, nothing cherry picked, no unfavorable result excluded. This is what let the project report, with real confidence, that the champion agent beats all three fixed time baselines on 53 of 60 independently random evaluations (88.3%).

Two properties of the environment make this methodology trustworthy rather than just elaborate: SUMO's seeded vehicle generation is genuinely deterministic (the same seed produces a bit for bit identical vehicle stream regardless of which controller is driving the signal, verified empirically across process restarts), and evaluation episodes now terminate as soon as the road is genuinely empty rather than running to a fixed time limit, which cut typical evaluation time by roughly seven to eight times with no change to the reported results.

B.12 Known Limitations and Future Work

- Single intersection scope: this project controls one intersection. Multi intersection coordination is unattempted future work.
- High traffic ceiling: performance near the intersection's physical demand capacity reflects queueing physics, not a perception limitation (ruled out directly by a V9 experiment with a richer, non saturating observation encoding, see the book, Section 5.3).
- Training is not reproducible run to run: two independent runs of the identical V8 recipe (V8 and V8_replicate) produced meaningfully different stability profiles. The leading hypothesis is the constant, non decaying entropy coefficient. This is a concrete, addressable item for future work, not a fully solved question.
- Computer vision perception (YOLO + DeepSORT), proposed in the original Phase A plan, was not implemented in this delivered scope. See the book, Chapter 4, for the full explanation. It remains the most natural next step for extending this project toward real world deployment.
- No formal, rigorous head to head comparison exists between the DQN agent and the fixed time baselines using the same seed verified methodology applied to PPO. Old_Versions/DQN_Agent/results/comparison_history.json contains real evaluation history but was not brought to the same level of statistical rigor.

B.13 Coding Conventions Observed in This Codebase

- Copy, don't mutate: a new agent version is a new folder, never an in place edit of a previous one's environment or training script. This preserves every earlier version as a working, comparable reference.
- Path resolution: every script computes _HERE = os.path.dirname(os.path.abspath(__file__)) and builds paths relative to it, rather than assuming a particular working directory. Shared modules (evaluate_models.py) are colocated as sibling files within PPO_Agent/scripts/ rather than imported via a separate src/ folder one level up, which is how the earlier, archived versions under Old_Versions/PPO/ still do it. This is a historical difference worth knowing about if you compare the two, not an inconsistency to "fix."
- Crash safety: any long running sweep writes its full plan before starting and appends results incrementally, so a kill or crash partway through loses nothing already computed and can be resumed.
- Structural constraints over penalties: wherever a behavior must never happen (switching an empty intersection, violating minimum/maximum green), the project moved from discouraging it with a reward penalty to preventing it outright via action masking, after direct evidence that penalties alone were unreliable. Apply the same principle to any new hard constraint.